

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG

Lê Minh Thông

THIẾT KẾ VÀ KIỂM THỬ BỘ NHỚ ĐỆM L2 THỰC HIỆN MESI
PROTOCOL TRONG COHERENCE MODEL

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN
NGÀNH KỸ THUẬT ĐIỆN TỬ-VIỄN THÔNG
CHUYÊN NGÀNH: ĐIỆN TỬ

Thành phố Hồ Chí Minh – Năm 2025

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỄN THÔNG

KHÓA LUẬN/ĐỒ ÁN TỐT NGHIỆP CỬ NHÂN
NGÀNH KỸ THUẬT ĐIỆN TỬ-VIỄN THÔNG
CHUYÊN NGÀNH: ĐIỆN TỬ

**TÊN ĐỀ TÀI: THIẾT KẾ VÀ KIỂM THỬ BỘ NHỚ ĐỆM L2 THỰC
HIỆN MESI PROTOCOL TRONG COHERENCE MODEL**

Họ và tên sinh viên: Lê Minh Thông

Mã số SV: 21200356

NGƯỜI HƯỚNG DẪN KHOA HỌC:

1. ThS. Nguyễn Duy Mạnh Thi
- 2.

Thành phố Hồ Chí Minh – Năm 2025

XÁC NHẬN HOÀN TẤT CHỈNH SỬA BÁO CÁO TỐT NGHIỆP

TÊN ĐỀ TÀI: THIẾT KẾ VÀ KIỂM THỬ BỘ NHỚ ĐỆM L2 THỰC HIỆN MESI PROTOCOL TRONG COHERENCE MODEL

Tên tiếng Anh: Design and Verification of an L2 Cache Implementing the MESI Protocol in the Coherence Model

Họ và tên sinh viên: Lê Minh Thông

Mã số sinh viên: 21200356

Hội đồng đánh giá họp ngày..... tháng năm.....

1. Chủ tịch:

2. Thư ký:

3. Ủy viên:

4. Ủy viên:

5. Ủy viên:

Người hướng dẫn: ThS. Nguyễn Duy Mạnh Thi

Xác nhận của người hướng dẫn:

LỜI CẢM ƠN

Lời đầu tiên em xin gửi lời cảm ơn chân thành đến thầy Nguyễn Duy Mạnh Thi đã tận tình đồng hành và hỗ trợ em trong thời gian thực hiện khóa luận vừa qua. Trong thời gian qua, thầy đã luôn tạo điều kiện và đưa ra nhiều góp ý đầy kinh nghiệm để em có thể hoàn thiện từng phần nội dung của bài luận này. Nhờ những buổi trao đổi thẳng thắn và động viên quý giá của thầy mà em có định hướng rõ ràng cũng như vững tinh thần trong những lúc khó khăn. Không chỉ kiến thức về chuyên môn mà thầy còn giúp em hiểu rõ hơn về quá trình thực hiện một dự án thực tế. Một lần nữa, em xin gửi lời cảm ơn đến thầy vì sự hướng dẫn tận tâm và động viên quý giá trong thời gian thực hiện vừa qua.

Tôi xin cam đoan những số liệu và kết quả nghiên cứu được trình bày trong khóa luận tốt nghiệp là trung thực. Tôi cam kết không sao chép nội dung hay kết quả của các nghiên cứu khác đã được công bố. Những tài liệu tham khảo được tôi sử dụng trong báo cáo đã được trích dẫn một cách đầy đủ, rõ ràng về nguồn gốc theo quy định.

Tôi xin chịu trách nhiệm nếu vi phạm các cam kết trên.

Họ tên sinh viên

Thông

Lê Minh Thông

TÓM TẮT KHÓA LUẬN

Đối tượng là bộ nhớ đệm L1 tích hợp trong nhân xử lý (Core) trong mô hình mạch lạc (Coherence Model), cần phải nhỏ, đơn giản và nhanh, nên không thể thực hiện MESI protocol. Khóa luận này tập trung vào thiết kế một bộ nhớ đệm L2 ở ngoài Core và nằm giữa L1 và Snoop Bus. Thiết kế này ngoài hỗ trợ MESI protocol cơ bản còn được mở rộng thêm các tính năng để đảm bảo tính mạch lạc cho bộ nhớ đệm L1. Ngoài ra bài luận còn xây dựng một môi trường UVM với đầy đủ các thành phần để gửi và nhận các tín hiệu trên Interface, sau đó đánh giá kết. Scoreboard để kiểm tra tính đúng đắn của thiết kế về mặt hành vi và Scoreboard để đảm bảo tính bao phủ của các kịch bản được kiểm tra. Kết quả kiểm thử cho thấy mạch đã hoạt động đúng với MESI protocol và đảm bảo được tính nhất quán trên bộ nhớ đệm L1.

DESIGN AND VERIFICATION OF AN L2 CACHE IMPLEMENTING THE MESI PROTOCOL IN THE COHERENCE MODEL

The target of this thesis is the L1 cache allocated inside the processor core in a coherence model. Because the L1 cache is limited in terms of size, simplicity, and speed, it's not available to implement the full MESI protocol directly. This thesis focuses on designing an external L2 cache that allocated between the L1 cache and the Snoop Bus. This L2 cache fully supports the MESI protocol and extended with additional features to help maintain data consistency for the L1 cache. A UVM verification environment is also built to verify a design. This includes all essential components to generate and monitor signals over the cache interface. The verification environment includes a scoreboard to check the correctness of the design's behavior, and coverage collectors to ensure that all scenarios are exercised during testing. The test results show that the circuit operates correctly according to the MESI protocol and maintains consistency in the L1 cache.

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Bối cảnh và lý do chọn đề tài.....	1
1.2 Ý tưởng và mục tiêu.....	2
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	4
2.1 Bộ nhớ đệm.....	4
2.1.1 Phân cấp các thiết bị lưu trữ	4
2.1.2 Cấu trúc bộ nhớ đệm.....	5
2.1.3 Kết quả tra cứu bộ nhớ	7
2.1.4 Phương pháp ánh xạ bộ nhớ	7
2.1.5 Thuật toán thay thế	8
2.2 MESI protocol.....	8
2.2.1 Giới thiệu mô hình Coherence và MESI protocol	8
2.2.2 Danh sách trạng thái của khối dữ liệu	9
2.2.3 Cơ chế ghi dữ liệu xuống bộ nhớ thấp hơn	10
2.2.4 Cơ chế cập nhập các bản sao trên các bộ nhớ đệm.....	10
2.2.5 Danh sách lệnh yêu cầu	10
2.2.6 Sự chuyển tiếp trạng thái	11
2.2.7 Tính bao hàm và tính độc quyền giữa các cấp bộ nhớ đệm	12
2.3 Môi trường kiểm định UVM.....	13
2.3.1 Giới thiệu môi trường UVM.....	13
2.3.2 Các thành phần cơ bản trong môi trường UVM.....	13
2.3.3 Scoreboard	14
2.3.4 Thống kê về mặt tính năng	15
CHƯƠNG 3: THIẾT KẾ BỘ NHỚ ĐỆM L2 THỰC HIỆN MESI PROTOCOL	16
3.1 Thiết kế phần cứng.....	16

3.1.1	Tham số cấu hình thiết kế.....	16
3.1.2	Mở rộng MESI protocol	16
3.1.3	Thực hiện Coherence protocol.....	18
3.1.4	Danh sách tín hiệu	20
3.1.5	Sơ đồ mạch	24
3.1.6	Bộ xử lý yêu cầu từ bộ nhớ đệm L1	28
3.1.7	Bộ xử lý yêu cầu từ Snoop Bus	30
3.1.8	Thuật toán thay thế PLRU	32
3.2	Kiểm định chức năng của thiết kế.....	33
3.2.1	Môi trường UVM.....	33
3.2.2	Dạng sóng tín hiệu	38
3.2.3	Kế hoạch kiểm thử.....	39
3.2.4	Mô phỏng kịch bản hợp lệ	43
3.2.5	Mô phỏng kịch bản không mong muốn.....	50
3.2.6	Kết quả và đánh giá	53
CHƯƠNG 4: KẾT LUẬN		56
4.1	Đánh giá kết quả.....	56
4.1.1	Ưu điểm	56
4.1.2	Nhược điểm	56
4.2	Định hướng phát triển	56

DANH MỤC TỪ VỰNG CHUYÊN NGÀNH

Multi-processor	Hệ thống với nhiều bộ xử lý cùng hoạt động song song với nhau.
Core	Nhân xử lý hay mỗi bộ xử lý trong hệ thống Multi-processor.
Cache	Bộ nhớ đệm, nằm giữa nhân xử lý và bộ nhớ chính, chứa một phần dữ liệu của bộ nhớ chính nhằm tăng tốc độ truy cập bộ nhớ.
Private cache	Bộ nhớ đệm mà dữ liệu của nó chỉ được truy cập bởi một nhân xử lý duy nhất.
Shared cache	Bộ nhớ đệm mà dữ liệu của nó có thể được truy cập bởi nhiều nhân xử lý khác nhau.
Coherence model	Mô hình dùng để đảm bảo tính nhất quán của dữ liệu trên tất cả bộ nhớ trong hệ thống Multi-processor.
Coherence protocol	Giao thức được dùng trong Coherence model.
Snoop bus	Đường Bus chung, kết nối các bộ nhớ đệm lại với nhau nhằm đảm bảo tính nhất quán trong toàn hệ thống.
Downstream	Chiều dữ liệu gửi từ bộ nhớ cấp cao hơn (gần nhân xử lý hơn) đến bộ nhớ cấp thấp hơn (gần bộ nhớ chính hơn).
Upstream	Chiều dữ liệu gửi từ bộ nhớ cấp thấp hơn (gần bộ nhớ chính hơn) đến bộ nhớ cấp cao hơn (gần nhân xử lý hơn).
Write back	Cơ chế ghi dữ liệu xuống bộ nhớ thấp hơn chỉ khi vùng địa đó bị trục xuất ra khỏi bộ nhớ đệm.
Write through	Cơ chế ghi dữ liệu xuống bộ nhớ thấp hơn ngay khi vùng địa chỉ đó được điều chỉnh.
Block	Thành phần lưu trữ trong bộ nhớ đệm, bao gồm các trường Tag, Data, State.
Index	Một phần của địa chỉ yêu cầu, dùng để xác định Block trong quá trình ánh xạ.

Tag	Phần còn lại của địa chỉ sau khi loại bỏ phần Index, dùng để xác định địa chỉ có tồn tại trong bộ nhớ đệm hay không.
Lookup	Quá trình tra cứu bộ nhớ để xem địa chỉ yêu cầu có tồn tại trong bộ nhớ đệm hay không.
Hit	Kết quả tra cứu cho thấy địa chỉ yêu cầu có tồn tại trong bộ nhớ đệm.
Fill	Kết quả tra cứu cho thấy địa chỉ yêu cầu không tồn tại trong bộ nhớ đệm, lưu dữ liệu mới vào Block trống.
Evict	Kết quả tra cứu cho thấy địa chỉ yêu cầu không tồn tại trong bộ nhớ đệm, trục xuất dữ liệu của một Block đã có chứa dữ liệu để lưu dữ liệu mới.
Way	Chỉ các Block khả thi trong quá trình ánh xạ dùng trong cấu trúc Ways Set Associative.
Replacement policy	Phương pháp xác định Block sẽ bị trục xuất tiếp theo khi không có bất cứ Way nào có khả năng lưu trữ dữ liệu mới.
Functional Coverage	Thông số chỉ mức độ bao phủ về mặt tính năng, của các kịch bản đã được thực hiện trong bài Test.

DANH SÁCH HÌNH ẢNH

Hình 1-1: Mô hình phân cấp bộ nhớ dùng trong bài luận.....	2
Hình 2-1: Phân cấp bộ nhớ	4
Hình 2-2: Quy ước chiều của Upstream và Downstream	5
Hình 2-3: Phân vùng địa chỉ yêu cầu	5
Hình 2-4: Cấu trúc vùng nhớ trong bộ nhớ đệm.....	6
Hình 2-5: So sánh phương pháp ánh xạ Direct Mapped và 2-Way Set Associative	7
Hình 2-6: Chuyển tiếp trạng thái trong MESI protocol	11
Hình 2-7: Mô hình bộ nhớ thỏa tính bao hàm	12
Hình 2-8: Mô hình bộ nhớ thỏa tính độc quyền.....	12
Hình 2-9: Các thành phần cơ bản trong môi trường UVM.....	13
Hình 2-10: Mô hình hóa Scoreboard	14
Hình 2-11: Quan sát Functional Coverage sau khi mô phỏng	15
Hình 3-1: Sự chuyển tiếp trạng thái trong thiết kế	19
Hình 3-2: Sơ đồ khối của thiết kế	24
Hình 3-3: Quy trình bắt tay trên mỗi kênh.....	25
Hình 3-4: Cấu trúc vùng nhớ trong thiết kế.....	26
Hình 3-5: Bộ xử lý yêu cầu từ bộ nhớ đệm L1	27
Hình 3-6: Sơ đồ thuật toán tra cứu bộ nhớ trong bộ xử lý yêu cầu từ bộ nhớ đệm L1	28
Hình 3-7: Chuyển tiếp trạng thái trong bộ xử lý yêu cầu từ bộ nhớ đệm L1	29
Hình 3-8: Sơ đồ thuật toán tra cứu bộ nhớ trong bộ xử lý yêu cầu từ Snoop Bus.....	30
Hình 3-9: Chuyển tiếp trạng thái trong bộ xử lý yêu cầu từ Snoop Bus	31
Hình 3-10: Thuật toán câu nhị phân trong PLRU.....	32
Hình 3-11: Môi trường UVM dùng trong bài luận	33
Hình 3-12: Khái quát hoạt động của Monitor trong bài luận bằng SystemVerilog	34
Hình 3-13: Monitor in ra Log file trong khi hoạt động	34
Hình 3-14: Mô hình hóa cấu trúc vùng nhớ bằng SystemVerilog	35
Hình 3-15: Kiểu dữ liệu của các trường trong Block bằng SystemVerilog	35
Hình 3-16: Các lệnh API giao tiếp với mô hình tham chiếu.....	35
Hình 3-17: Ví dụ quá trình kiểm tra hành vi của Scoreboard.....	36
Hình 3-18: Kết quả đối chiếu mô hình tham chiếu và thiết kế	37

Hình 3-19: Tổng kết quá trình kiểm thử	37
Hình 3-20: Dạng sóng tín hiệu của thiết kế khi xử lý yêu cầu	38
Hình 3-21: Quy trình hoạt động của Scoreboard của thiết kế	41
Hình 3-22: Ví dụ Coverpoint cho một kịch bản không hợp lệ	42
Hình 3-23: Theo dõi kết quả Coverage bằng công cụ mô phỏng	43
Hình 3-24: Dạng sóng kênh SDREQ và SURSP của ví dụ	44
Hình 3-25: Dạng sóng trên kênh CDREQ và CURSP của ví dụ	44
Hình 3-26: Kết quả đối chiếu của ví dụ	44
Hình 3-27: Dạng sóng trên kênh CDREQ và CURSP của ví dụ	45
Hình 3-28: Kết quả đối chiếu của ví dụ	45
Hình 3-29: Dạng sóng trên kênh CUREQ và CDRSP của ví dụ	46
Hình 3-30: Dạng sóng kênh SDREQ và SURSP của ví dụ	46
Hình 3-31: Dạng sóng trên kênh CDREQ và CURSP của ví dụ	47
Hình 3-32: Kết quả đối chiếu của ví dụ	47
Hình 3-33: Dạng sóng trên kênh SUREQ và SDRSP của ví dụ	48
Hình 3-34: Kết quả đối chiếu của ví dụ	48
Hình 3-35: Dạng sóng trên kênh CUREQ và CDRSP của ví dụ	48
Hình 3-36: Dạng sóng trên kênh SDREQ và SURSP của ví dụ	49
Hình 3-37: Dạng sóng trên kênh SUREQ và SDRSP của ví dụ	49
Hình 3-38: Kết quả đối chiếu của ví dụ	49
Hình 3-39: Dạng sóng trên kênh CDREQ và CURSP của ví dụ	50
Hình 3-40: Kết quả đối chiếu của ví dụ	50
Hình 3-41: Dạng sóng trên kênh CDREQ và CURSP của ví dụ	51
Hình 3-42: Kết quả đối chiếu của ví dụ	51
Hình 3-43: Dạng sóng trên kênh SUREQ và SDRSP của ví dụ	52
Hình 3-44: Kết quả đối chiếu của ví dụ	52
Hình 3-45: Kết quả Coverage trong bài Test về MESI protocol	53
Hình 3-46: Kết quả thống kê của Scoreboard trong bài Test về MESI protocol	53
Hình 3-47: Kết quả Coverage trong bài Test về các trường hợp không mong muốn	54
Hình 3-48: Kết quả thống kê của Scoreboard trong bài Test về các trường hợp không mong muốn	54
Hình 3-49: Tổng kết Coverage trong tất cả Test	55

DANH SÁCH BẢNG

Bảng 2-1: Danh sách trạng thái của MESI protocol	9
Bảng 2-2: Danh sách loại yêu cầu trong MESI protocol	10
Bảng 3-1: Các tham số cấu hình trong thiết kế.....	16
Bảng 3-2: Danh sách trạng thái dữ liệu trong thiết kế.....	18
Bảng 3-3: Danh sách các kênh tín hiệu.....	20
Bảng 3-4: Danh sách tín hiệu.....	21
Bảng 3-5: Danh sách mã hóa các tín hiệu.....	22
Bảng 3-6: Danh sách các kịch bản hợp lệ của thiết kế	39
Bảng 3-7: Danh sách các kịch bản không mong muốn trong thiết kế	40
Bảng 3-8: Coverage cho các kịch bản hợp lệ	42

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh và lý do chọn đề tài

Với sự phát triển của các thiết bị công nghệ hiện nay, các yêu cầu về hiệu năng đặt lên bộ xử lý ngày càng lớn. Các bộ xử lý hiện đại yêu cầu tốc độ phải nhanh hơn, tiêu hao ít năng lượng và diện tích phải nhỏ để có thể tích hợp được nhiều linh kiện khác.

Để giải quyết các yêu cầu trên, nhiều phương pháp đã được thực hiện và đã mang lại hiệu quả tích cực. Các bộ xử lý hiện đại không nhất thiết phải truy cập xuống bộ nhớ chính (DRAM), mà có thể thông bộ nhớ đệm (Cache) như là nơi lưu trữ trung gian, để thuận tiện trong việc truy xuất dữ liệu.

Ngoài ra, mô hình đa nhân xử lý (Multi-processor) có thể chia bộ xử lý thành nhiều nhân (Core) hoạt động song song nhau, giải quyết tốt yêu cầu tăng tốc độ, thay thế cho phương pháp tăng tần số Clock đang bị đi bão hòa, do gặp khó khăn trong kỹ thuật và vật lý.

Tuy nhiên việc chia ra nhiều Core lại phát sinh ra một vấn đề khác là phải đảm bảo tính nhất quán của dữ liệu (data consistency) giữa các bộ đệm. Để đạt được yêu cầu trên, mô hình mạch lạc (Coherence model) được ra đời, với nhiều giao thức (Coherence protocol), trong đó MESI là một giao thức phổ biến với khả năng cân bằng giữa sự đơn giản và tính hiệu quả.

Trong đề tài này, đối tượng nghiên cứu là một hệ thống Multi-processor hỗ trợ MESI protocol. Với mỗi nhân (Core) phải đạt yêu cầu về tối ưu diện tích, các bộ nhớ đệm được đặt trong Core (như bộ nhớ đệm) phải nhỏ gọn và có tốc độ nhanh. Do đó, bộ nhớ đệm L1 không thể kết nối trực tiếp với Bus để truy xuất dữ liệu do độ trễ cao, và không thể hỗ trợ các giao thức phức tạp như MESI vì sự phức tạp, yêu cầu nhiều diện tích.

Từ vấn đề trên, đề tài hướng việc tìm giải pháp sao cho bộ nhớ đệm L1 có thể tuân thủ tính nhất quán dựa trên MESI protocol, có tốc độ truy xuất dữ liệu nhanh, nhưng vẫn phải đảm bảo tính nhỏ gọn để có thể tích hợp vào trong Core.

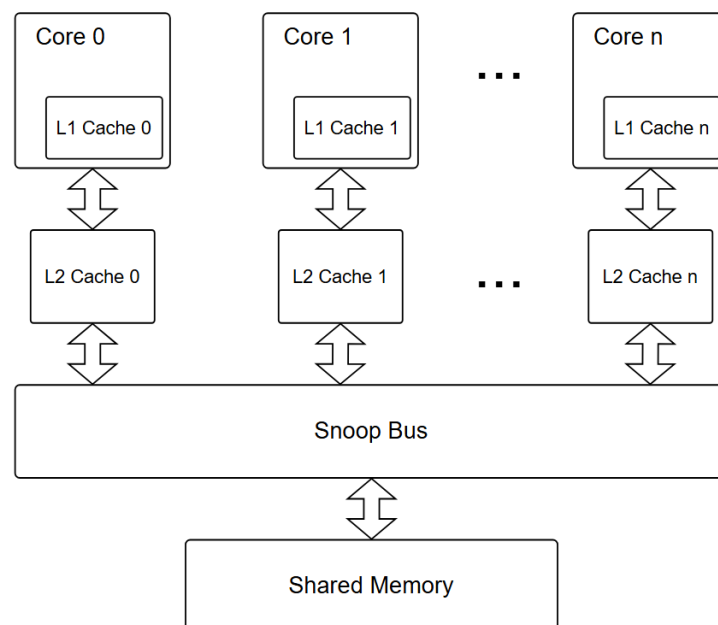
1.2 Ý tưởng và mục tiêu

Ý tưởng là sẽ thiết kế một bộ nhớ đệm L2 được đặt ngoài Core để có thể có kích thước lưu trữ lớn hơn, từ đó, tăng tỉ lệ tìm thấy thông tin (Hit) mà không phải truy xuất xuống tận bộ nhớ chính (Main Memory). Ngoài ra, vì nằm ngoài Core nên ít bị giới hạn bởi diện tích hơn, từ đó, hỗ trợ nhiều tính năng hơn.

Bộ nhớ đệm chung, hay chia sẻ (Shared Cache) là bộ nhớ đệm mà dữ liệu của nó có thể được chia sẻ cho nhiều Core khác, nhưng vì bộ nhớ đệm L1 không hỗ trợ MESI protocol nên nó không thể đảm bảo tính nhất quán trên các bộ nhớ đệm L1 khác nếu như bộ nhớ của L2 được chia sẻ. Do đó, giống như bộ nhớ đệm L1, bộ nhớ đệm L2 cũng được xây dựng như là bộ nhớ đệm riêng (Private Cache), với dữ liệu của nó chỉ được truy cập bởi một Core duy nhất.

Ngoài ra, để đảm bảo tính nhất quán trên toàn hệ thống, phải đảm bảo rằng tất cả thông tin trong L1 phải tồn tại trong bộ nhớ đệm L2, từ đó dữ liệu trong cả hai bộ nhớ đều sẽ được theo dõi trên Snoop Bus. Điều kiện này còn gọi là tính bao hàm (Inclusivity Property).

Mô hình phân cấp giữa bộ nhớ đệm L1, L2 và Snoop Bus được sử dụng trong bài luận này được thể hiện như sau:



Hình 1-1: Mô hình phân cấp bộ nhớ dùng trong bài luận

Theo như hình trên, bộ nhớ đệm L1 và L2 là bộ nhớ đệm riêng (Private Cache), trong khi đó bộ nhớ chung (Shared Cache) có thể là bộ nhớ đệm L3 hoặc bộ nhớ chính tùy hệ thống.

Về cách quản lý bộ nhớ trong L2, thiết kế sẽ dùng phương pháp ánh xạ 4-Way Set Associative và thuật toán thay thế là PLRU (Pseudo Least Recently Used) nhằm cân bằng hiệu suất và độ phức tạp của thiết kế.

Ngoài giao thức MESI chuẩn, bộ nhớ đệm L2 cần phải mở rộng thêm các tính năng để đảm bảo tính mạch lạc trên bộ nhớ đệm L1. Trong bài luận này, bộ nhớ đệm L2 hỗ trợ thêm một trạng thái của dữ liệu là trạng thái Migrated và một loại yêu cầu mới từ bộ nhớ đệm L1 là Mark Dirty.

Lệnh Mark Dirty thông báo rằng vùng địa chỉ yêu cầu đã được điều chỉnh (giá trị khác so với bộ nhớ chính) và dữ liệu mới nhất đang ở trong bộ nhớ đệm L1. Khi nhận được yêu cầu Mark Dirty, bộ nhớ đệm L2 sẽ chuyển trạng thái của vùng địa chỉ được yêu cầu thành Migrated (đã di chuyển).

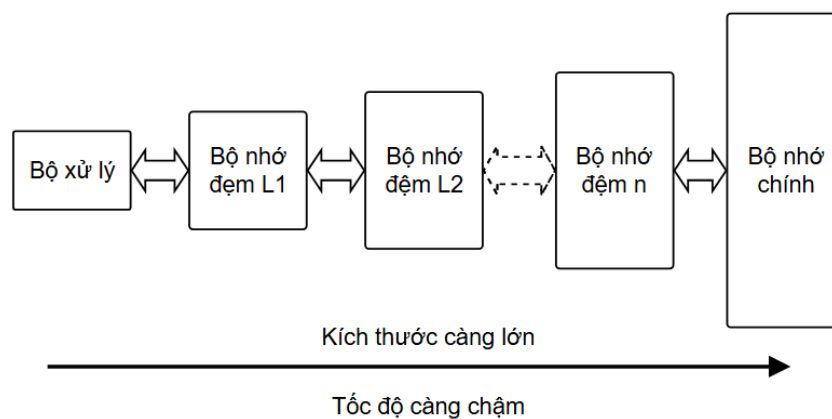
Bài luận này còn có xây dựng một môi trường UVM (Universal Verification Methodology) nhằm đảm bảo thiết kế hoạt động đúng như yêu cầu đề ra, hay đặc tính kỹ thuật (Specification). Trong môi trường gồm có các thành phần như Sequencer, Driver, Monitor để gửi và nhận các tín hiệu kích thích và kết quả trả về. Ngoài ra, còn có Scoreboard để đánh giá tính đúng đắn của thiết kế về mặt hành vi, dùng cơ chế Read Backdoor giúp truy cập sâu vào các thanh ghi trong thiết kế mà không cần thông qua các tín hiệu Input và Output, Coverage được dùng để đánh giá tính bao quát của các tín hiệu kích thích.

Từ các ý tưởng trên, mục tiêu của bài luận này như sau:

- Đảm bảo thiết kế hoạt động đúng với giao thức MESI trong trên cả hai phía là bộ nhớ đệm L1 và Snoop Bus.
- Đảm bảo dữ liệu được lưu trữ và truy xuất đúng theo các trường Index, Way, Tag và Data.
- Đảm bảo tính bao hàm (Inclusivity Property).
- Xây dựng môi trường UVM để đánh giá tính đúng đắn của thiết kế dựa theo yêu cầu của bài luận.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT**2.1 Bộ nhớ đệm****2.1.1 Phân cấp các thiết bị lưu trữ**

Một vấn đề ảnh hưởng đến hiệu năng của bộ xử lý là thời gian truy cập bộ nhớ. Thời gian truy cập bộ nhớ lớn làm kéo dài thời gian thực hiện lệnh của bộ xử lý. [1]Ở các thiết bị phức tạp, bộ xử lý thường không được kết nối trực tiếp với bộ nhớ chính (Main Memory) mà phải đi qua nhiều thiết bị trung gian, điều này dẫn đến thời gian đọc và ghi dữ liệu vào bộ nhớ là rất lớn. [2]Bộ nhớ đệm (Cache) ra đời, với mục đích là lưu trữ một phần dữ liệu từ bộ nhớ chính và được đặt gần với bộ xử lý hơn. Nếu dữ liệu yêu cầu có tồn tại trong bộ nhớ đệm, bộ nhớ đệm sẽ trả dữ liệu phản hồi về cho bộ xử lý, mà không cần phải truy cập đến bộ nhớ chính. Điều này có thể giảm đáng kể thời gian truy cập bộ nhớ khi hoạt động.

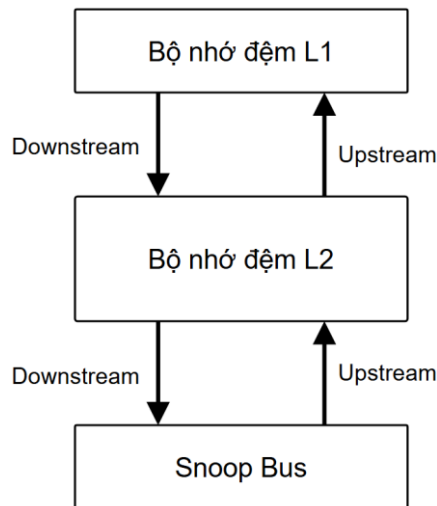


Hình 2-1: Phân cấp bộ nhớ

Trong hình trên, giữa bộ xử lý và bộ nhớ chính có nhiều bộ nhớ đệm nằm giữa như trạm trung gian. [3]Bộ nhớ đệm cấp cao hơn (Higher level Cache) được quy ước là nằm gần với bộ xử lý hơn, ngược lại, bộ nhớ đệm cấp thấp hơn (Lower level Cache) sẽ nằm gần bộ nhớ chính hơn. Cấp bộ nhớ đệm càng thấp thì kích thước càng lớn, ngược lại tốc độ truy cập sẽ càng chậm.

[4]Nếu bộ xử lý muốn đọc dữ liệu, đầu tiên nó sẽ gửi yêu cầu đến bộ nhớ đệm L1, nếu bộ nhớ đệm L1 có tồn tại bản sao của địa chỉ được yêu cầu, nó sẽ gửi dữ liệu phản hồi về cho bộ xử lý, ngược lại, bộ xử lý L1 sẽ tiếp tục gửi yêu cầu đọc dữ liệu xuống bộ nhớ L2, lần lượt như vậy đến bộ nhớ chính nếu không có bất kỳ bộ nhớ đệm nào tồn tại bản sao của vùng địa chỉ được yêu cầu.

Vì mỗi bộ nhớ đệm đều kết nối với hai bộ nhớ đệm khác, nằm ở hai chiều ngược nhau, với mỗi chiều đều phải nhận được yêu cầu và có thể gửi yêu cầu ngược lại cho bộ nhớ đó. Do đó, để phân biệt chiều của kênh tín hiệu, dựa vào hướng của kênh tín hiệu so với bộ nhớ chính, chiều của kênh được chia thành hai loại, với kênh thuận (Downstream) và kênh nghịch (Upstream).

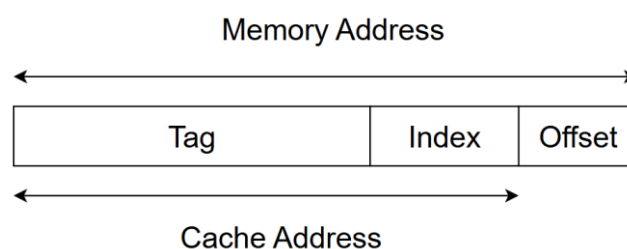


Hình 2-2: Quy ước chiều của Upstream và Downstream

Dựa vào hình trên chúng ta quy ước rằng, [5]kênh tín hiệu nào có chiều hướng đến bộ nhớ chính thì được gọi là kênh thuận (Downstream), ngược lại, có chiều hướng đến bộ xử lý thì được gọi là kênh nghịch (Upstream).

2.1.2 Cấu trúc bộ nhớ đệm

2.1.2.1 Phân vùng địa chỉ yêu cầu



Hình 2-3: Phân vùng địa chỉ yêu cầu

Trước tiên chúng ta cần phân biệt được địa chỉ bộ nhớ (Memory Address) và địa chỉ Cache (Cache Address). [1]Địa chỉ bộ nhớ dùng để xác định dữ liệu trong bộ nhớ, với mỗi giá trị tương ứng với một Byte dữ liệu. Địa chỉ Cache dùng để xác định Block trong Coherence Model, với mỗi giá trị tương ứng với một Block dữ liệu. Địa chỉ Cache là địa chỉ bộ nhớ

được loại bỏ trường Offset (với độ rộng của trường Offset là logaric cơ số hai của số Byte trong Block). Trong phạm vi khóa luận, khi đề cập đến “địa chỉ”, chúng ta ngầm hiểu đó là địa chỉ Cache.

Khi nhận được yêu cầu, [4]bộ Cache sẽ phân tách địa chỉ thành các trường Index và Tag, sau đó truy xuất vào bộ nhớ trong Cache để xác định vùng địa chỉ có tồn tại trong Cache hay không. Trường Index dùng để xác định vị trí của Block trong Cache với độ rộng bằng logaric cơ số hai của số Block có trong Cache. Trường Tag là phần còn lại của địa chỉ sau khi loại bỏ phần Index. Ngoài ra, với mỗi Block còn có một bit thể hiện rằng dữ liệu trong Block này là có nghĩa (Valid Block).

2.1.2.2 Cấu trúc bộ nhớ

[4]Bộ nhớ của bộ nhớ đệm (Cache) được chia thành nhiều phần nhỏ, gọi là Block (hay Cache Line). Mỗi Block chứa các trường Data và Tag dùng để xác định vùng địa chỉ được yêu cầu có tồn tại trong bộ nhớ đệm hay không.

Index	Valid	Tag	Data	
0x0				Block 0
0x1				Block 1
0x2				Block 2
0x3				Block 3
0x4				Block 4
0x5				Block 5
0x6				Block 6
0x7				Block 7

Hình 2-4: Cấu trúc vùng nhớ trong bộ nhớ đệm

Trường Data là nơi lưu trữ dữ liệu của Block, kích thước có thể là 16 Byte, 32 Byte hoặc 64 Byte (thường là 64 Byte). Trường Index dùng để đánh dấu số thứ tự của Block trong bộ nhớ. Tag là nơi lưu trữ phần còn lại của địa chỉ, sau khi loại bỏ trường Index. Trường Valid dùng để xác định rằng dữ liệu trong Block này là có nghĩa.

2.1.3 Kết quả tra cứu bộ nhớ

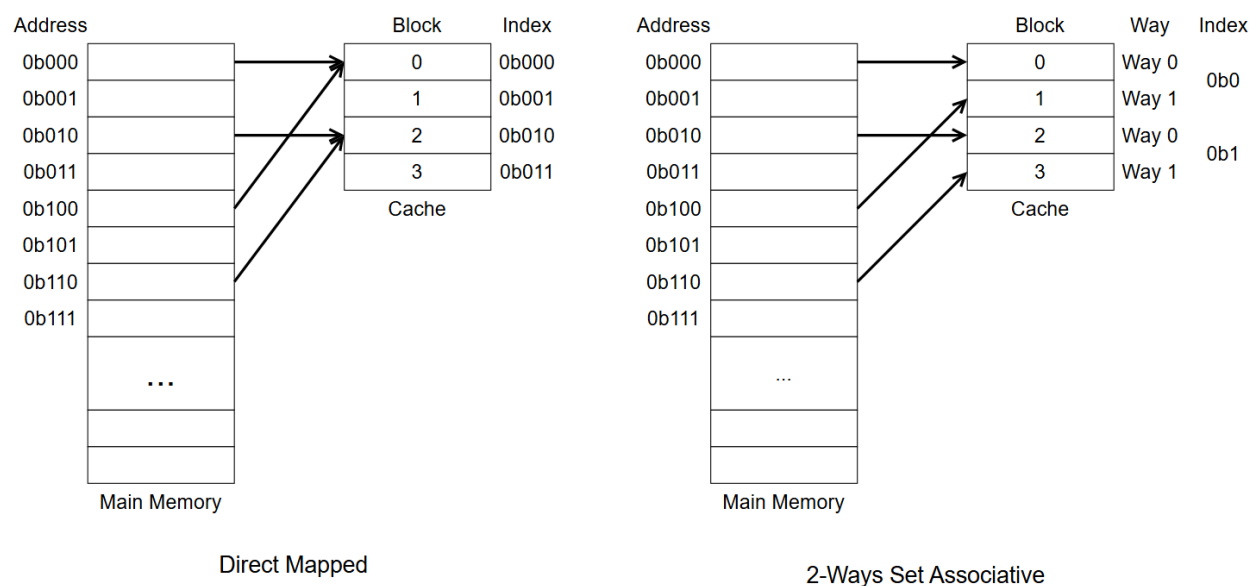
[4] Khi một yêu cầu được gửi đến, kết quả khi tra cứu địa chỉ yêu cầu trong bộ nhớ có thể là:

- Hit: dữ liệu của địa chỉ được yêu cầu có tồn tại trong bộ nhớ
- Miss: dữ liệu của địa chỉ được yêu cầu không tồn tại trong bộ nhớ, do đó phải truy xuất dữ liệu từ Snoop Bus. Từ đây tiếp tục được chia thành hai loại.
 - Fill: tồn tại một Block trống (Invalid) (với giá trị Index tương ứng) để lưu trữ thông tin trả về.
 - Evict: Block với Index tương ứng đã được dùng để chứa dữ liệu, do đó phải truy xuất Block đó để lưu trữ thông tin trả về.

2.1.4 Phương pháp ánh xạ bộ nhớ

[6] Phương pháp ánh xạ bộ nhớ (Cache Mapping) có hai loại, thứ nhất là Direct Mapped – mỗi giá trị Index sẽ được ánh xạ đến duy nhất một Block, phương pháp hai là Set Associative – mỗi giá trị của Index sẽ được ánh xạ đến cơ sở dữ liệu hai Block, với mỗi Block còn được gọi là Way. Ví dụ, nếu một giá trị Index có thể ánh xạ đến hai Block khác nhau, ta nói đó là 2-Ways Set Associative.

So sánh giữa Direct Mapped và 2-Ways Set Associative được thể hiện như sau:



Hình 2-5: So sánh phương pháp ánh xạ Direct Mapped và 2-Way Set Associative

Theo hình trên, chúng ta có thể thấy trong 2-Ways Set Associative, có hai Blocks (Ways) thỏa mãn để lưu trữ dữ liệu mới, do đó phương pháp Set Associative có nhiều lựa chọn để cấp phát bộ nhớ hơn Direct Mapped, dẫn đến số lần phải trục xuất Block ít hơn và tăng tỉ lệ Hit, nhưng đồng thời cũng làm cho thiết kế trở nên phức tạp hơn.

2.1.5 Thuật toán thay thế

2.1.5.1 Giới thiệu về thuật toán thay thế

[7]Trong trường hợp bộ nhớ đệm cần lưu trữ dữ liệu mới vào bộ nhớ và tất cả Way với Index tương ứng đều đã được dùng để lưu trữ dữ liệu. Lúc này, cần có một thuật toán để xác định nên trục xuất Block nào, mà có khả năng bị truy cập trong tương lai là thấp nhất, để có thể duy trì tỉ lệ Hit. Thuật toán đó gọi là thuật toán thay thế (Replacement Policy).

2.1.5.2 Thuật toán LRU và PLRU

[8]Đối với thuật toán LRU (Least Recently Used), Block với lần truy cập cuối cùng là xa nhất sẽ bị trục xuất khỏi bộ nhớ. Vì cần phải theo dõi chính xác thứ tự truy cập cuối của từng Way trong mỗi Index, nên LRU đòi hỏi một lượng lớn tài nguyên phần cứng.

Để giải quyết vấn đề này, trong các bộ nhớ đệm hiện đại, thuật toán PLRU (Pseudo-LRU) được sử dụng phổ biến hơn nhờ tính đơn giản, ít tốn tài nguyên hơn. Tuy nhiên, vài trường hợp PLRU sẽ trục xuất Block không thỏa theo LRU dẫn đến tỉ lệ Hit thấp hơn so với LRU.

2.2 MESI protocol

2.2.1 Giới thiệu mô hình Coherence và MESI protocol

[4]Một phương pháp truyền thống để tăng tốc độ của bộ xử lý là tăng tần số Clock để có thể xử lý được nhiều phép toán hơn trong một giây. Phương pháp này hiệu quả khi tiến trình (Process) của thời bấy giờ còn cao, tuy nhiên, do giới hạn về kỹ thuật và vật lý nên việc tiếp tục giảm tiến trình là không còn bền vững. Thêm nữa, tần số hoạt động càng cao đi chung với mức tiêu hao năng lượng càng lớn. Để giải quyết vấn đề trên, các bộ xử lý hiện đại thường dùng mô hình xử lý đa nhân (Multi-processor), với ý tưởng là sẽ có nhiều nhân xử lý (Core) cùng hoạt động song song với nhau. Từ đó, nhiều nhiệm vụ được thực hiện cùng lúc, dẫn đến tăng tốc độ xử lý.

[4]Tuy nhiên việc chia ra nhiều Core lại phát sinh ra một vấn đề khác là phải đảm bảo tính nhất quán của dữ liệu (data consistency) giữa các bộ đệm. Để đạt được yêu cầu trên, mô

hình mạch lạc (coherence model) được ra đời, với nhiều giao thức (coherence protocol), trong đó MESI là một giao thức phổ biến với khả năng cân bằng giữa sự đơn giản và tính hiệu quả.

2.2.2 Danh sách trạng thái của khối dữ liệu

[4] Trong bộ nhớ đệm cơ bản, chỉ cần một bit để thể hiện rằng dữ liệu trong Block là có nghĩa, tuy nhiên, trong Coherence Model, cần phát triển thêm nhiều trạng thái hơn để đảm bảo tính nhất quán trên từng Block.

Khi bộ nhớ đệm nhận một yêu cầu, nó sẽ dựa vào loại yêu cầu, kết quả tra cứu bộ nhớ và trạng thái của vùng địa chỉ để thực hiện hành động tương ứng.

Danh sách trạng thái của MESI bao gồm:

Bảng 2-1: Danh sách trạng thái của MESI protocol

Trạng thái	Nhãn	Miêu tả
I	Invalid	- Dữ liệu của địa chỉ này không tồn tại trong bộ nhớ đệm này - Nếu cần đọc dữ liệu, phải truy xuất thông tin từ bộ nhớ cấp thấp hơn
E	Exclusve	- Dữ liệu của địa chỉ này có tồn tại trong bộ nhớ đệm này - Dữ liệu trong Block trùng với bộ nhớ chính - Chỉ có bộ nhớ đệm này chứa bản sao của của địa chỉ này
S	Shared	- Dữ liệu của địa chỉ này có tồn tại trong bộ nhớ đệm - Dữ liệu trong Block trùng với bộ nhớ chính - Có thể tồn tại nhiều hơn một bộ nhớ đệm có bản sao của của địa chỉ này
M	Modified	- Dữ liệu của địa chỉ này có tồn tại trong bộ nhớ đệm - Dữ liệu trong Block khác so với bộ nhớ chính - Chỉ có bộ nhớ đệm này chứa bản sao của của địa chỉ này

2.2.3 Cơ chế ghi dữ liệu xuống bộ nhớ thấp hơn

[9] Có hai cơ chế được dùng để bộ nhớ đệm ghi dữ liệu xuống bộ nhớ thấp hơn là ghi trì hoãn (Write Back) và ghi trực tiếp (Write Through). Trong cơ chế Write Back, hành động ghi dữ liệu của một Block xuống bộ nhớ thấp hơn chỉ diễn ra khi Block đó sắp bị trục xuất và dữ liệu đã được điều chỉnh (khác so với bộ nhớ chính). Với cơ chế Write Through, dữ liệu của Block sẽ được ghi xuống bộ nhớ thấp hơn ngay khi có bị điều chỉnh trên Block đó. Trong MESI protocol, thiết kế sẽ dùng cơ chế Write Back để giảm thiểu số yêu cầu được tạo ra trên Snoop Bus.

2.2.4 Cơ chế cập nhập các bản sao trên các bộ nhớ đệm

[10] Có hai cơ chế để đảm bảo bản sao trong tất cả bộ nhớ đệm trong hệ thống nhận được phiên bản mới nhất là ghi vô hiệu hóa (Write Invalidate) và ghi cập nhập (Write Update). Trong cơ chế Write Invalidate, khi bản sao trên một bộ nhớ đệm bất kỳ được điều chỉnh, dữ liệu bây giờ không còn giống với bộ nhớ chính, bộ nhớ đệm đó sẽ gửi lệnh vô hiệu hóa (Invalidate) đến các bộ nhớ đệm khác để xóa bản sao của vùng địa chỉ đó. Với cơ chế Write Update, nó sẽ gửi lệnh cập nhập (Update) cùng với dữ liệu mới nhất của vùng địa chỉ đó đến các bộ nhớ đệm còn lại. Trong MESI protocol, thiết kế sẽ dùng cơ chế Write Invalidate để tiết kiệm băng thông trên Snoop Bus.

2.2.5 Danh sách lệnh yêu cầu

[4] MESI protocol hỗ trợ các lệnh sau ở trên hai hướng là bộ xử lý và Snoop Bus để đảm bảo tính nhất quán trong mô hình Coherence:

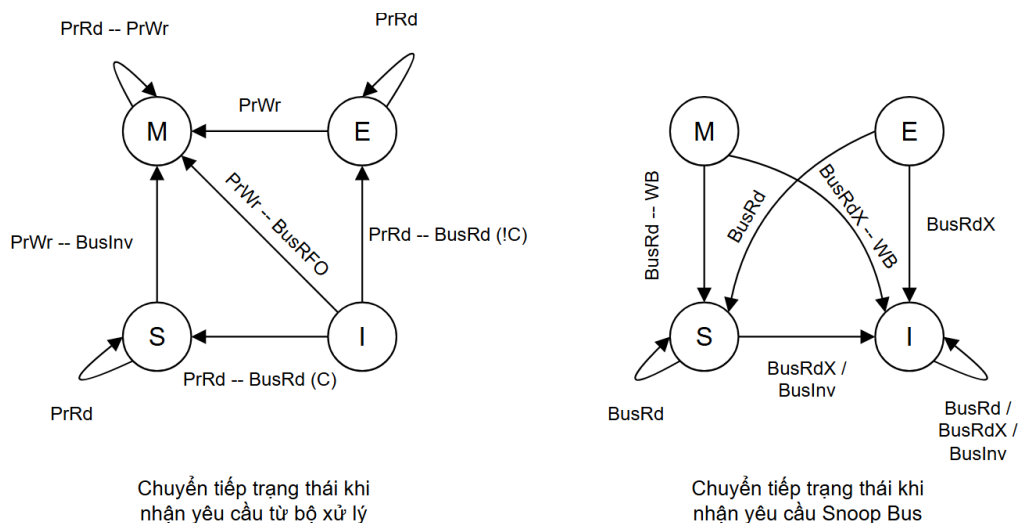
Bảng 2-2: Danh sách loại yêu cầu trong MESI protocol

Thiết bị yêu cầu	Yêu cầu	Nhãn	Miêu tả
Bộ xử lý	Read	PrRD	Yêu cầu đọc dữ liệu vùng địa chỉ
	Write	PrWr	Yêu cầu ghi dữ liệu vào vùng địa chỉ
Snoop Bus	Read	BusRd	Yêu cầu đọc dữ liệu vùng địa chỉ
	RFO	BusRFO	Read for ownership – yêu cầu đọc dữ liệu và xóa bản sao vùng địa chỉ
	Invalidate	BusInv	Vô hiệu hóa bản sao của vùng địa chỉ
Bộ nhớ đệm	Write Back	WB	Ghi dữ liệu xuống bộ nhớ thấp hơn

2.2.6 Sự chuyển tiếp trạng thái

Khi nhận được yêu cầu từ bộ xử lý hoặc Snoop Bus, tùy vào loại yêu cầu, kết quả tra cứu bộ nhớ và trạng thái hiện tại của vùng địa chỉ được yêu cầu, bộ nhớ đệm có thể tạo ra các yêu cầu phát sinh trên Snoop Bus để truy xuất dữ liệu, cũng như đảm bảo tính nhất quán trong mô hình Coherence.

Sự chuyển tiếp trạng thái của mỗi Block và khởi tạo yêu cầu phát sinh được thể hiện như sau:



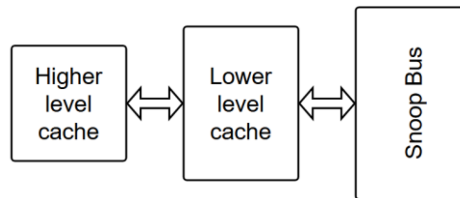
Hình 2-6: Chuyển tiếp trạng thái trong MESI protocol

Trong hình trên, có các quy ước sau:

- Các ô tròn nối với đuôi của mũi tên thể hiện trạng thái của Block ngay thời điểm vừa nhận được yêu cầu.
- Các ô tròn nối với đầu của mũi tên thể hiện trạng thái của Block sau khi yêu cầu được thực hiện thành công.
- Mỗi mũi tên ứng với một trường hợp chuyển trạng thái khác nhau.
- Yêu cầu ban đầu được gửi từ bộ xử lý hoặc Snoop Bus sẽ đứng trước dấu “—”, và yêu cầu phát sinh (nếu có) được khởi tạo bởi bộ nhớ đệm L2 đứng sau dấu “—”.
- Các trường hợp khác nhau nhưng có cùng sự chuyển đổi trạng thái (cùng trên một mũi tên) sẽ được phân biệt bằng dấu “/”.
- Chữ “C” (hay Cache) thể hiện địa chỉ yêu cầu có tồn tại trong bộ nhớ đệm khác.

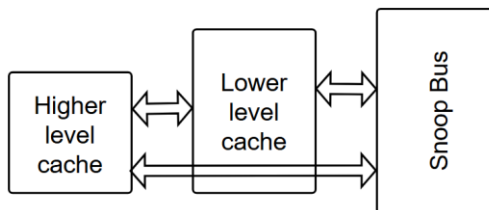
2.2.7 Tính bao hàm và tính độc quyền giữa các cấp bộ nhớ đệm

[11] Tính bao hàm (Inclusivity Property) quy định rằng dữ liệu của các bộ nhớ đệm cấp thấp hơn (được đặt ở gần mới bộ nhớ chính hơn) phải bao hàm hết dữ liệu của các bộ nhớ đệm cấp cao hơn (được đặt ở gần với bộ xử lý hơn). Ví dụ, bản sao của các vùng địa chỉ tồn tại trong bộ nhớ đệm L1 cũng phải tồn tại trong bộ nhớ đệm L2.



Hình 2-7: Mô hình bộ nhớ thỏa tính bao hàm

[11] Ngược lại với tính bao hàm, tính độc quyền (Exclusivity Property) quy định rằng dữ liệu của các bộ nhớ đệm cấp thấp hơn (được đặt ở gần mới bộ nhớ chính hơn) không được tồn tại trong các bộ nhớ đệm cấp cao hơn (được đặt ở gần với bộ xử lý hơn). Ví dụ, bản sao của các vùng địa chỉ tồn tại trong bộ nhớ đệm L2 không được tồn tại trong bộ nhớ đệm L1.



Hình 2-8: Mô hình bộ nhớ thỏa tính độc quyền

Vì dữ liệu ở bộ nhớ đệm cấp thấp hơn không xuất hiện tại trong các bộ nhớ đệm cấp cao hơn, nên tính độc quyền sẽ hiệu quả trong khả năng lưu trữ. Tuy nhiên, do cả hai cấp đều phải hỗ trợ giao thức MESI nên sẽ gây khó khăn cho bộ nhớ đệm cấp cao hơn nếu cần phải đảm bảo tính đơn giản, lúc này tính bao hàm lại đem lại hiệu quả cao hơn.

Trong phạm vi khóa luận, vì bộ nhớ đệm L1 không kết nối trực tiếp với Snoop Bus và phải thông qua bộ nhớ đệm L2. Do cần phải đảm tính mạch lạc giữa bộ nhớ đệm L1 với các bộ nhớ khác, giữa bộ nhớ đệm L1 và L2 phải đảm bảo tính bao hàm (Inclusivity Property).

2.3 Môi trường kiểm định UVM

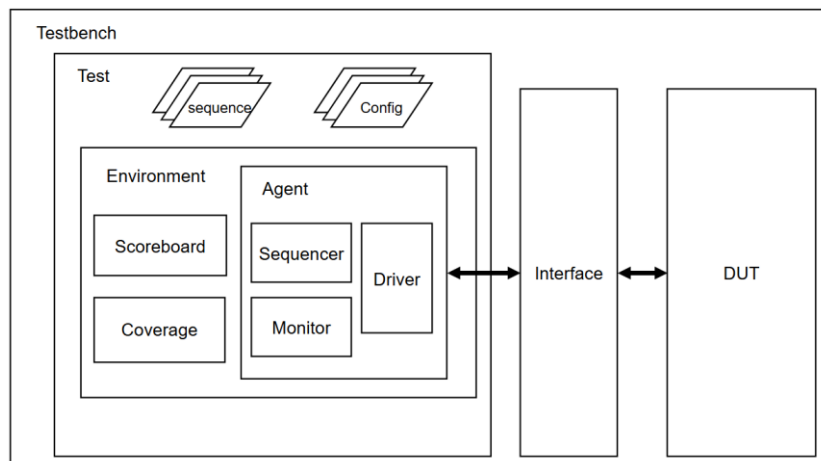
2.3.1 Giới thiệu môi trường UVM

[12]UVM (Universal Verification Methodology) là framework dựa trên ngôn ngữ SystemVerilog, được sử dụng rộng rãi cho việc kiểm thử các thiết kế vi mạch. Trong các môi trường kiểm thử lớn, việc xây dựng từ đầu đến cuối là rất khó khăn và không tối ưu, do đó chúng ta cần một framework để hỗ trợ các mẫu có sẵn

Thêm nữa, trong giai đoạn kiểm thử, sẽ có rất nhiều người và nhóm làm việc với nhau, việc sử dụng một quy chuẩn chung sẽ tăng hiệu quả công việc hơn nhiều lần.

UVM hỗ trợ các Classes và các Methods cho phép người dùng dễ dàng tái sử dụng để tạo ra các môi trường kiểm thử khác nhau. Testbench Components là các thành phần cấu thành lên một Testbench như Driver, Monitor, Scoreboard, Testcase, ... Transaction chứa các tín hiệu kiểm tra để gửi vào thiết kế. Các giai đoạn hoạt động của từng Components được đồng bộ hóa bằng cơ chế Phase. Messaging và Reporting để gửi các thông báo ra cho người dùng. Configuration dùng để truyền dữ liệu lên Database hoặc các Components trong môi trường.

2.3.2 Các thành phần cơ bản trong môi trường UVM



Hình 2-9: Các thành phần cơ bản trong môi trường UVM

Testbench là module lớn nhất trong môi trường UVM, dùng để chứa tất cả các thành phần như Environment, Interface và DUT (Design Under Test). Các tín hiệu Clock và Reset sẽ được tạo ở trong Testbench và được truyền vào Interface, từ đó tạo sự đồng bộ trên toàn môi trường.

Để đảm bảo rằng thiết kế hoạt động đúng như mong đợi, cần tạo ra nhiều Test khác nhau để kiểm thử tất cả tính năng của mạch. Mỗi Test có thể thay đổi về Topology, kích bản, tín hiệu kích thích hoặc trình tự hoạt động tùy vào mục đích kiểm thử.

UVM Environment được khởi tạo trong Test, là nơi chứa các thành phần như Agent, Scoreboard, Coverage,... Trong đó, Agent là thành phần sẽ nhận các kích bản của Test, sau đó sẽ gửi và bắt tín hiệu trên Interface, hay giao tiếp với DUT.

Cơ chế bắt tay (Handshake) giữa Sequence, Sequencer và Driver giúp người dùng có thể kiểm soát được các kích bản kiểm thử được tạo ra và gửi đi. Sequence được tạo bởi Test, mang các cấu hình và dữ liệu kích thích. Sequencer như là một thiết bị phân phối, đảm nhận nhiệm vụ lần lượt cho phép các Sequence được giao tiếp với Driver. Cuối cùng Driver sẽ truyền các dữ liệu kích của xuống Interface.

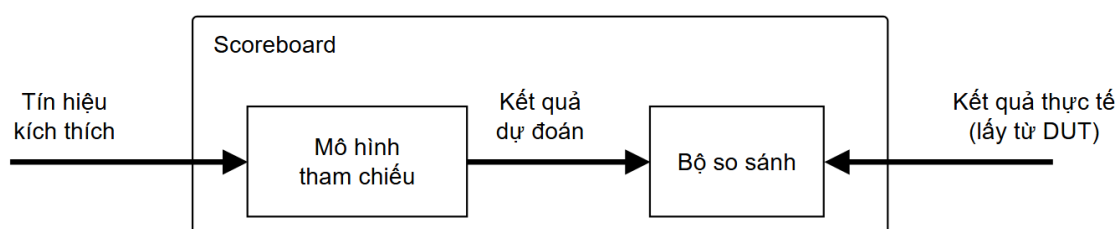
Monitor là thành phần dùng để bắt các tín hiệu của thiết kế trên Interface, đóng gói và gửi cho các thành phần khác để kiểm tra tính đúng đắn của thiết kế.

2.3.3 Scoreboard

Scoreboard sẽ mô phỏng lại hành vi của thiết kế dựa theo đặc điểm kỹ thuật (Specification), từ đó kiểm tra xem thiết kế đã hoạt động đúng mong đợi hay chưa.

Để kiểm tra tính đúng đắn của thiết kế, trong Scoreboard sẽ tồn tại một mô hình tham chiếu (Reference Model). Mô hình tham chiếu sẽ mô hình hóa lại hành vi của thiết kế dựa trên đặc tính kỹ thuật, nhưng được viết bằng ngôn ngữ cấp cao hơn để thuận tiện trong việc mô hình hóa và tái sử dụng.

Trong quá trình mô phỏng, Scoreboard sẽ nhận dữ liệu trên Interface từ Monitor, từ đó so sánh các thông tin đó với mô hình tham chiếu, nếu hai dữ liệu khác nhau sẽ báo lỗi cho người dùng.



Hình 2-10: Mô hình hóa Scoreboard

2.3.4 Thống kê về mặt tính năng

[13] Thống kê về mặt tính năng (Functional Coverage) là thông số để đảm bảo rằng các tính năng của thiết kế đã được kiểm tra. Không như Scoreboard kiểm tra về hành vi của thiết kế, thống kê về mặt tính năng (Functional Coverage) kiểm tra tính bao phủ của các kịch bản (Scenario) đã được kiểm thử.

Trong trường hợp thiết kế lớn và phức tạp, cần tạo nhiều Test để kiểm tra từng tính năng của thiết kế, chúng ta có thể chia Coverage thành nhiều phần tương ứng với từng bài Test. Sau khi mô phỏng tất cả các Test, chúng ta có thể gộp các tất cả các Coverage thành phần lại thành một Coverage tổng quát.

Dưới đây là ví dụ về việc quan sát Functional Coverage trong khi chạy mô phỏng:

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/cache_units_pkg/cache_cov_cdreq_c		56.1%				
TxPB CG_CDREQ_MESI_COV	cache_cov_...	77.5%	100	77.5%		✓
CVP CG_CDREQ_MESI_COV::CP_STATE_COV	cache_cov_...	80.0%	100	80.0%		✓
CVP CG_CDREQ_MESI_COV::CP_CDREQ_OP_COV	cache_cov_...	75.0%	100	75.0%		✓
CVP CG_CDREQ_MESI_COV::CP_LOOKUP_COV	cache_cov_...	100.0%	100	100.0%		✓
CVP CG_CDREQ_MESI_COV::CP_CUREQ_OP_COV	cache_cov_...	50.0%	100	50.0%		✓
CVP CG_CDREQ_MESI_COV::CP_CDREQ_RSP_COV	cache_cov_...	100.0%	100	100.0%		✓
CVP CG_CDREQ_MESI_COV::CP_SDREQ_OP_COV	cache_cov_...	66.6%	100	66.6%		✓
CVP CG_CDREQ_MESI_COV::CP_SURSP_RSP_COV	cache_cov_...	100.0%	100	100.0%		✓
CVP CG_CDREQ_MESI_COV::CP_CURSP_RSP_COV	cache_cov_...	100.0%	100	100.0%		✓
CROSS CG_CDREQ_MESI_COV::CROSS_SDREQ_OP_X_SURSP_RSP_COV	cache_cov_...	75.0%	100	75.0%		✓
CROSS CG_CDREQ_MESI_COV::CROSS_CDREQ_OP_X_LOOKUP_X_STATE	cache_cov_...	28.5%	100	28.5%		✓
INST \cache_units_pkg::cache_cov_cdreq_c::CG_CDREQ_MESI_COV		77.5%	100	77.5%		✓
TxPB CG_CDREQ_CORNER_CASE_COV	cache_cov_...	56.6%	100	56.6%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_CURSP_RSP_COV	cache_cov_...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_L1_FETCH_HIT_MIGRATED_COV	cache_cov_...	50.0%	100	50.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_WB_X_HIT_NOT_MIGRATED_COV	cache_cov_...	33.3%	100	33.3%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_MD_X_MISS_COV	cache_cov_...	50.0%	100	50.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_WB_X_MISS_COV	cache_cov_...	50.0%	100	50.0%		✓
INST \cache_units_pkg::cache_cov_cdreq_c::CG_CDREQ_CORNER_CASE_COV		56.6%	100	56.6%		✓
TxPB CG_CDREQ_BLK_COV	cache_cov_...	34.1%	100	34.1%		✓
/cache_units_pkg/cache_cov_sureq_c		54.7%				
TxPB CG_SUREQ_MESI_COV	cache_cov_...	76.2%	100	76.2%		✓
TxPB CG_SUREQ_CONER_CASE_COV	cache_cov_...	33.3%	100	33.3%		✓

Hình 2-11: Quan sát Functional Coverage sau khi mô phỏng

Một hạn chế của Functional Coverage là tính chuẩn xác của thống kê phụ thuộc vào tính chính xác của người viết Coverage. Ví dụ, nếu thiết kế có mười tính năng, và người viết coverage vô tình chỉ viết được cho tám tính năng. Nếu thiết kế hoạt động đúng với tám tính năng đã được liệt kê và sai hết ở hai tính năng còn lại, trong trường hợp này, mạch chạy chưa đúng mới đặt tính kỹ thuật, nhưng trong Coverage thì thống kê được rằng mạch đã thực hiện và chính xác hết các tính năng.

CHƯƠNG 3: THIẾT KẾ BỘ NHỚ ĐỆM L2 THỰC HIỆN MESI PROTOCOL

3.1 Thiết kế phần cứng

3.1.1 Tham số cấu hình thiết kế

Trong các thiết kế phần cứng, các tham số (Parameter) giúp người dùng có thể cấu hình các đặc tính của thiết kế mà không cần phải điều chỉnh trực tiếp vào mã nguồn, giúp tăng tính linh hoạt và khả năng tái sử dụng của thiết kế.

Trong phạm vi khóa luận, thiết kế có hỗ trợ các tham số sau:

Bảng 3-1: Các tham số cấu hình trong thiết kế

Tên	Miêu tả
ADDR_WIDTH	Độ rộng của địa chỉ bộ nhớ (Memory Address)
DATA_WIDTH	Độ rộng của trường Data trong một Block
NUM_BLK	Số lượng Block trong bộ nhớ đệm L2

3.1.2 Mở rộng MESI protocol

3.1.2.1 Lý do mở rộng

Trong giao thức MESI, các cấp của bộ nhớ đệm đều phải hỗ trợ giao thức MESI để đảm bảo tính mạch lạc với các bộ nhớ đệm khác trong cùng hoặc khác cấp, kể cả bộ nhớ đệm L1.

Tuy nhiên, trong đối tượng nghiên cứu của bài luận, bộ nhớ đệm L1 cần phải đơn giản và tối ưu diện tích nên không thể hỗ trợ giao thức MESI, từ đó, không thể kết nối trực tiếp với Snoop Bus mà phải thông qua bộ nhớ đệm L2 như là thiết bị trung gian. Từ vấn đề trên, cần phải mở rộng bộ nhớ đệm L2 để đảm bảo tính nhất quán cho bộ nhớ đệm L1.

Trong khóa luận này, ngoài hỗ trợ đầy đủ các tính năng của phương thức MESI, bộ nhớ đệm L2 hỗ trợ thêm một trạng thái dữ liệu mới đó là trạng thái “đã di chuyển” (Migrated) và một loại yêu cầu mới từ bộ nhớ đệm L1 là Mark Dirty.

3.1.2.2 Yêu cầu Mark Dirty

Nếu bộ xử lý ghi dữ liệu xuống vùng địa chỉ đã tồn tại trong bộ nhớ đệm L1 (Write Hit), áp dụng theo cơ chế Write Back, bộ nhớ đệm L1 chưa ghi dữ liệu xuống bộ nhớ đệm L2 ngay để tiết kiệm băng thông. Tuy nhiên, điều này có khả năng làm mất tính nhất quán của hệ thống vì bộ nhớ đệm L2 vẫn nghĩ rằng địa chỉ trên chưa được chiều chỉnh, dẫn đến lỗi trong các hoạt động chuyển trạng thái hay gửi yêu cầu phát sinh.

Do đó, để đảm bảo tính nhất quán, bộ nhớ đệm L1 phải gửi lệnh Mark Dirty cho bộ nhớ đệm L2 để thông báo rằng, vùng địa chỉ trên đã được chỉnh sửa và dữ liệu mới nhất đang nằm trong bộ nhớ đệm L1.

3.1.2.3 Trạng thái dữ liệu Migrated

Trạng thái Migrated (đã di chuyển) là trạng thái mở rộng từ trạng thái Modified (đã chỉnh sửa) trong giao thức MESI, thông báo rằng:

- Bản sao của vùng địa chỉ có tồn tại trong bộ nhớ đệm
- Không có bản sao của vùng địa chỉ này ở bất kỳ bộ nhớ đệm khác
- Dữ liệu của vùng địa chỉ đã được chỉnh sửa (khác) so với bộ nhớ chính.
- Tuy nhiên, khác với trạng thái Modified, dữ liệu mới nhất của vùng địa chỉ tương ứng đang nằm trong bộ nhớ đệm L2, trong trạng thái Migrated, dữ liệu mới nhất lại nằm trong bộ nhớ đệm L1.

3.1.2.4 Tương tác giữa trạng thái Migrated và lệnh Mark Dirty

Các tương tác giữa trạng thái Migrated và lệnh Mark Dirty được thể hiện như sau:

- Lệnh Mark Dirty không mang theo dữ liệu có nghĩa, do đó, bộ nhớ đệm L2 chỉ thay đổi trạng thái và phải giữ nguyên các trường Data và Tag đối với Block tương ứng.
- Nếu bộ xử lý tiếp tục ghi dữ liệu vào địa chỉ trên vào bộ nhớ đệm L1. Không bắt buộc rằng, bộ nhớ đệm L1 gửi lại lệnh Mark Dirty xuống bộ nhớ đệm L2.
- Một khi bộ nhớ đệm L2 gửi yêu cầu đọc dữ liệu vùng địa chỉ có trạng thái là Migrated cho L1. Bộ nhớ đệm L1 phải gửi phản hồi với dữ liệu mới nhất, đồng thời chuyển trạng thái của vùng địa chỉ đó sang trạng thái Invalid, nếu yêu cầu là Read, hoặc Shared, nếu yêu cầu là Read for Ownership.

3.1.3 Thực hiện Coherence protocol

3.1.3.1 Danh sách trạng thái của khối dữ liệu

Trong phạm vi khóa luận này, ngoài hỗ trợ các trạng thái của giao thức MESI, thiết kế cần hỗ trợ thêm trạng thái mới là Migrated để đảm bảo tính nhất quán cho bộ nhớ đệm L1.

Các trạng thái của mỗi Block trong thiết kế như sau:

Bảng 3-2: Danh sách trạng thái dữ liệu trong thiết kế

Trạng thái	Miêu tả
Invalid	Bản sao của địa chỉ này không tồn tại trong bộ nhớ đệm
Exclusive	- Bản sao của địa chỉ này có tồn tại trong bộ nhớ đệm - Chỉ có bộ nhớ đệm hiện tại chứa bản sao của địa chỉ này - Dữ liệu bộ nhớ đệm trùng với trong bộ nhớ chính
Shared	- Bản sao của địa chỉ này có tồn tại trong bộ nhớ đệm - Có thể tồn tại nhiều nhớ đệm có bản sao của địa chỉ này - Dữ liệu trong bộ nhớ đệm trùng với trong bộ nhớ chính.
Modified	- Bản sao của địa chỉ này có tồn tại trong bộ nhớ đệm - Chỉ có bộ nhớ đệm hiện tại chứa bản sao của địa chỉ này - Dữ liệu trong bộ nhớ đệm không trùng với bộ nhớ chính - Dữ liệu mới nhất đang ở bộ nhớ đệm L2
Migrated	- Bản sao của địa chỉ này có tồn tại trong bộ nhớ đệm - Chỉ có bộ nhớ đệm hiện tại chứa bản sao của địa chỉ này - Dữ liệu trong bộ nhớ đệm không trùng với bộ nhớ chính - Dữ liệu mới nhất đang ở bộ nhớ đệm L1

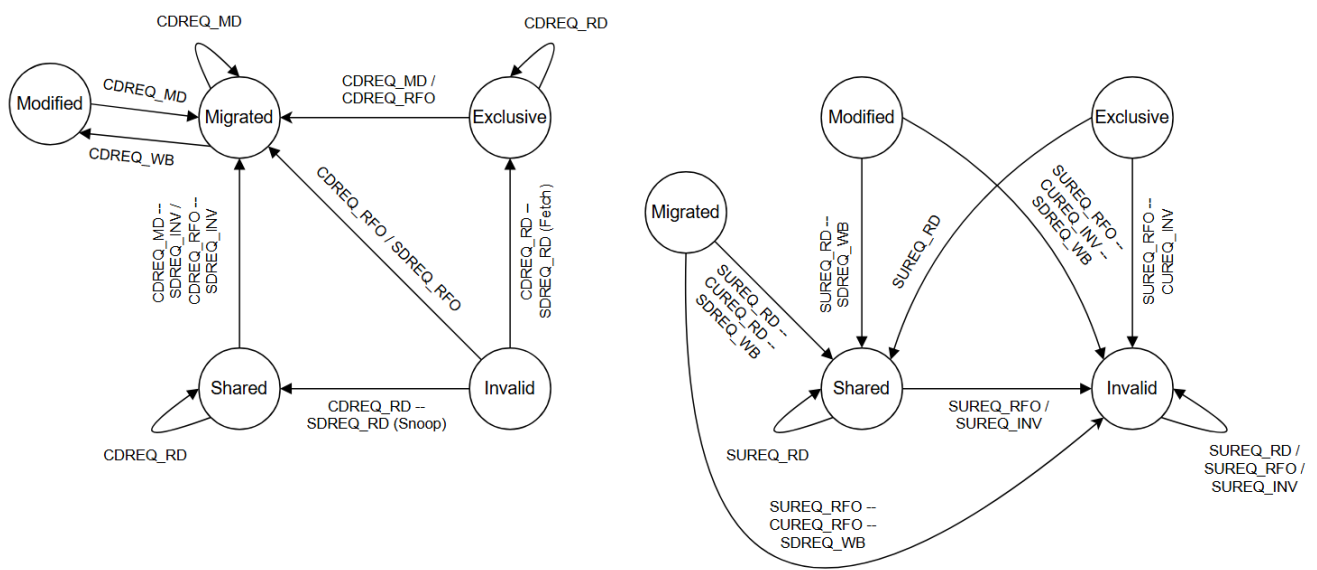
Trạng thái Migrated cho phép dữ liệu mới nhất của vùng địa chỉ tương ứng, có thể tồn tại trong bộ nhớ đệm L1 cho đến khi nó được trục xuất hoặc được đọc bởi L2. Việc này làm giảm số lượng yêu cầu được gửi giữa L1 và L2, nhưng vẫn đảm bảo tính nhất quán trên toàn bộ hệ thống.

3.1.3.2 Danh sách lệnh yêu cầu

3.1.3.3 Sự chuyển tiếp của các trạng thái

Khi nhận được yêu cầu từ bộ nhớ đệm L1 hoặc Snoop Bus, tùy vào loại yêu cầu, kết quả tra cứu bộ nhớ và trạng thái hiện tại của vùng địa chỉ được yêu cầu, bộ nhớ đệm có thể tạo ra các yêu cầu phát sinh trên Snoop Bus hoặc bộ nhớ đệm L1 để truy xuất dữ liệu, cũng như đảm bảo tính nhất quán trong mô hình Coherence.

Sự chuyển tiếp trạng thái của dữ liệu trong thiết kế được thể hiện như sau:



Hình 3-1: Sự chuyển tiếp trạng thái trong thiết kế

Trong hình trên, có các quy ước sau:

- Các ô tròn nối với đuôi của mũi tên thể hiện trạng thái của Block ngay thời điểm vừa nhận được yêu cầu.
- Các ô tròn nối với đầu của mũi tên thể hiện trạng thái của Block sau khi yêu cầu được thực hiện thành công.
- Mỗi mũi tên ứng với một trường hợp chuyển trạng thái khác nhau.
- Yêu cầu ban đầu được gửi từ bộ nhớ đệm L1 hoặc Snoop Bus sẽ đứng trước dấu “—”, và yêu cầu phát sinh (nếu có) được khởi tạo bởi bộ nhớ đệm L2 đứng sau dấu “—”.
- Các trường hợp khác nhau nhưng có cùng sự chuyển đổi trạng thái (cùng trên một mũi tên) sẽ được phân biệt bằng dấu “/”.

3.1.4 Danh sách tín hiệu**3.1.4.1 Danh sách các kênh**

Danh sách các kênh (Channel) tín hiệu trong thiết kế được miêu tả như sau:

Bảng 3-3: Danh sách các kênh tín hiệu

Giao tiếp với	Tên kênh	Thiết bị gửi	Miêu tả
Bộ nhớ đệm L1	CDREQ	L1 Cache	Cache downstream request. Dùng để gửi yêu cầu ban đầu từ bộ nhớ đệm L1
	CURSP	L2 Cache	Cache upstream response. Dùng để trả phản hồi của yêu cầu được gửi trên kênh CDREQ
	CUREQ	L2 Cache	Cache upstream request. Dùng để gửi yêu cầu phát sinh, được tạo ra từ các yêu cầu ban đầu trên kênh CDREQ hoặc kênh SUREQ
	CDRSP	L1 Cache	Cache downstream response. Dùng để trả phản hồi của yêu cầu được gửi trên kênh CUREQ
Snoop Bus	SUREQ	Snoop Bus	Snoop upstream request. Dùng để gửi yêu cầu ban đầu từ Snoop Bus
	SDRSP	L2 Cache	Snoop downstream response. Dùng để trả phản hồi của yêu cầu được gửi trên kênh SUREQ
	SDREQ	L2 Cache	Snoop downstream request. Dùng để gửi yêu cầu phát sinh, được tạo ra từ các yêu cầu ban đầu trên kênh CDREQ hoặc kênh SUREQ
	SURSP	Snoop Bus	Snoop upstream response. Dùng để trả phản hồi của yêu cầu được gửi trên kênh SDREQ

Trong bảng trên, chúng ta thấy tên của mỗi kênh được quy ước:

- Tên thiết bị được kết nối, với kênh Cx là kết nối với bộ nhớ đệm L1 và Sx là kết nối với Snoop Bus.
- Chiều của kênh tín hiệu, với xDx là chiều Downstream và xUx là Upstream.
- Nhiệm vụ của kênh tín hiệu, với xREQ là gửi tín hiệu yêu cầu và xRSP là gửi tín hiệu phản hồi.

3.1.4.2 Danh sách các tín hiệu

Danh sách tín hiệu trong mỗi kênh được miêu tả như sau:

Bảng 3-4: Danh sách tín hiệu

Channel	Tên	Hướng	Độ rộng	Miêu tả
System	Clk	Input	1	Tín hiệu Clock. Tất cả tín hiệu được lấy mẫu vào cạnh lên của tín hiệu Clock.
	Rst_n	Input	1	Tín hiệu Reset tích cực mức thấp.
CDREQ	Valid	Input	1	Thông báo các tín hiệu có nghĩa.
	Ready	Output	1	Thông báo sẵn sàng nhận tín hiệu
	Op	Input	3	Tín hiệu loại yêu cầu
	Addr	Input	SADDR_WIDTH	Tín hiệu địa chỉ
	Data	Input	DATA_WIDTH	Tín hiệu dữ liệu
CURSP	Valid	Output	1	Thông báo các tín hiệu có nghĩa.
	Ready	Input	1	Thông báo sẵn sàng nhận tín hiệu
	Rsp	Output	2	Tín hiệu loại phản hồi
	Data	Output	DATA_WIDTH	Tín hiệu dữ liệu
CUREQ	Valid	Output	1	Thông báo các tín hiệu có nghĩa.
	Ready	Input	1	Thông báo sẵn sàng nhận tín hiệu
	Op	Output	2	Tín hiệu loại yêu cầu
	Addr	Output	SADDR_WIDTH	Tín hiệu địa chỉ
CDRSP	Valid	Input	1	Thông báo các tín hiệu có nghĩa.
	Ready	Output	1	Thông báo sẵn sàng nhận tín hiệu
	Rsp	Input	2	Tín hiệu loại phản hồi
	Data	Input	DATA_WIDTH	Tín hiệu dữ liệu
SUREQ	Valid	Input	1	Thông báo các tín hiệu có nghĩa.
	Ready	Output	1	Thông báo sẵn sàng nhận tín hiệu
	Op	Input	2	Tín hiệu loại yêu cầu
	Addr	Input	SADDR_WIDTH	Tín hiệu địa chỉ
SDRSP	Valid	Output	1	Thông báo các tín hiệu có nghĩa.

	Ready	Input	1	Thông báo sẵn sàng nhận tín hiệu
	Rsp	Output	2	Tín hiệu loại phản hồi
	Data	Output	DATA_WIDTH	Tín hiệu dữ liệu
SDREQ	Valid	Output	1	Thông báo các tín hiệu có nghĩa.
	Ready	Input	1	Thông báo sẵn sàng nhận tín hiệu
	Op	Output	3	Tín hiệu loại yêu cầu
	Addr	Output	SADDR_WIDTH	Tín hiệu địa chỉ
	Data	Output	DATA_WIDTH	Tín hiệu dữ liệu
SURSP	Valid	Input	1	Thông báo các tín hiệu có nghĩa.
	Ready	Output	1	Thông báo sẵn sàng nhận tín hiệu
	Rsp	Input	3	Tín hiệu loại phản hồi
	Data	Input	DATA_WIDTH	Tín hiệu dữ liệu

Các tín hiệu handshake như Valid, Ready, và tín hiệu điều khiển như địa chỉ, dữ liệu, loại yêu cầu hoặc phản hồi chỉ có hiệu lực đối với kênh (channel) mà nó đang thuộc về, không tác động đến các kênh khác.

Ví dụ, tín hiệu Valid của kênh CDREQ không thể hiện rằng các tín hiệu của kênh CDRSP là có nghĩa.

3.1.4.3 Mã hóa các tín hiệu

Bảng mã hóa của các tín hiệu được miêu tả như sau:

Bảng 3-5: Danh sách mã hóa các tín hiệu

Tên	Giá trị	Nhãn	Miêu tả
CDREQ_Op	0b00	RD	Read. Yêu cầu đọc dữ liệu vùng địa chỉ
	0b01	RFO	Read for ownership. Yêu cầu đọc dữ liệu và xóa bản sao vùng địa chỉ
	0b10	WB	Write back. Yêu cầu ghi dữ liệu vào bộ nhớ cấp thấp hơn
	0b11	MD	Mark dirty. Thông báo rằng vùng địa chỉ này đã được chỉnh sửa, và dữ liệu mới nhất đang ở trên bộ nhớ đệm L1

CURSP_Rsp	0b0	OKAY	Giao dịch thành công
	0b1	ERROR	Giao dịch có lỗi xảy ra
CUREQ_Op	0b00	RD	Read. Yêu cầu đọc dữ liệu vùng địa chỉ
	0b01	RFO	Read for ownership. Yêu cầu đọc dữ liệu và xóa bản sao vùng địa chỉ
	0b10	INV	Invalidate. Yêu cầu xóa bản sao của vùng địa chỉ
CDRSP_Rsp	0b00	OKAY	Giao dịch thành công
	0b01	ERROR	Giao dịch có lỗi xảy ra
SUREQ_Op	0b00	RD	Read. Yêu cầu đọc dữ liệu vùng địa chỉ
	0b01	RFO	Read for ownership. Yêu cầu đọc dữ liệu và xóa bản sao vùng địa chỉ
	0b10	INV	Invalidate. Yêu cầu xóa bản sao của vùng địa chỉ
SDRSP_Rsp	0b00	OKAY	Giao dịch thành công
	0b01	INV	Invalid. Không tồn tại bản sao của vùng địa chỉ trong bộ nhớ đệm
	0b10	ERROR	Giao dịch có lỗi xảy ra
SDREQ_Op	0b00	RD	Read. Yêu cầu đọc dữ liệu vùng địa chỉ
	0b01	RFO	Read for ownership. Yêu cầu đọc dữ liệu và xóa bản sao vùng địa chỉ
	0b10	INV	Invalidate. Yêu cầu xóa bản sao của vùng địa chỉ
	0b11	WB	Write back. Yêu cầu ghi dữ liệu vào bộ nhớ cấp thấp hơn
SURSP_Rsp	0b00	OKAY	Giao dịch thành công
	0b01	FETCH	Dữ liệu trả về lấy từ bộ nhớ thấp hơn
	0b10	SNOOP	Dữ liệu trả về lấy từ bộ nhớ cùng cấp
	0b11	ERROR	Giao dịch có lỗi xảy ra

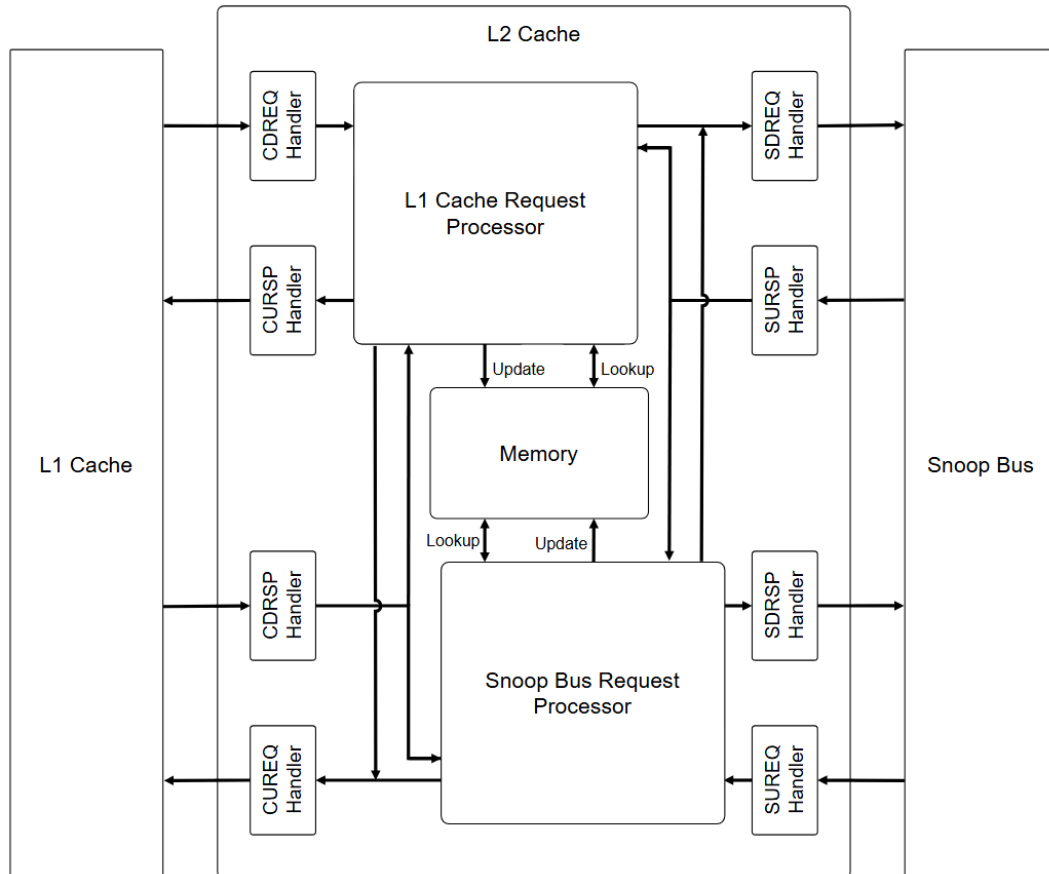
Các giá trị mã hóa không được gán với bất kỳ nhãn nào sẽ được hiểu là chưa dùng đến hoặc dự trữ (Reserve). Nếu thiết bị nhận được tín hiệu với giá trị trên sẽ có lỗi xảy ra.

Ví dụ giá trị 0b11 sẽ được hiểu là dự trữ (Reserve) vì không được gán với một ký nhãn có nghĩa nào.

3.1.5 Sơ đồ mạch

3.1.5.1 Sơ đồ khối

Sơ đồ khối của thiết kế được thể hiện như sau:



Hình 3-2: Sơ đồ khối của thiết kế

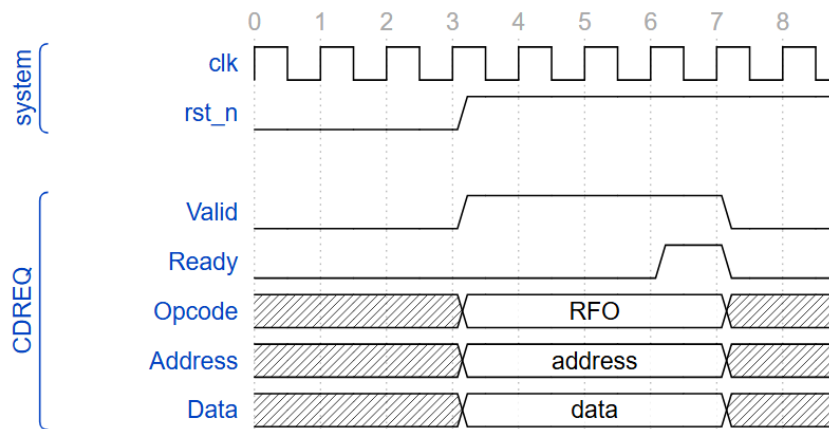
Theo hình trên, chúng ta có thể thấy ba thiết bị được kết nối với nhau là bộ nhớ đệm L1, bộ nhớ đệm L2 và Snoop Bus. Khối bên trái là bộ nhớ đệm riêng L1 (Private Cache), với bối cảnh của bài luận là phải đơn giản và nhỏ gọn để có thể tích hợp vào nhân xử lý (Core). Snoop Bus là đường Bus dùng để kết nối các bộ nhớ đệm cùng và khác cấp với lại với nhau, đảm bảo tính nhất quán trong mô hình Coherence. Ở giữa là bộ nhớ đệm L2, thiết kế được thảo luận chính trong bài luận này. Ngoài hỗ trợ đầy đủ tính năng của chuẩn MESI protocol, thiết kế này còn phát triển thêm các tính năng để đảm bảo tính nhất quán của bộ nhớ đệm L1 trong Coherence Model.

Trong bộ nhớ đệm L2 gồm có các mạch quản lý quy trình bắt tay (Handshake Handler) cho mỗi kênh, vùng nhớ (Memory) và hai module dùng để xử lý yêu cầu từ bộ nhớ đệm L1 (L1 Cache Request Processor) và từ Snoop Bus (Snoop Bus Cache Request Processor).

3.1.5.1.1 Mạch quản lý quy trình bắt tay

Quy trình bắt tay (Handshake) giúp đảm bảo tính chính xác và đồng bộ của dữ liệu được gửi, đồng thời cung cấp khả năng chủ động cho thiết bị nhận trong mỗi lần giao dịch.

Quy trình bắt tay được thể hiện ở hình sau:



Hình 3-3: Quy trình bắt tay trên mỗi kênh

Ở hình trên là một ví dụ của quy trình bắt tay trên kênh CDREQ, trình tự diễn ra như sau:

- Quá trình bắt tay bắt đầu tại Clock T3, lúc này tín hiệu Valid được tích cực và các thông tin trên các tín hiệu loại yêu cầu (Opcode), địa chỉ (Address) và dữ liệu (Data) được xem là có nghĩa.
- Tại Clock T4, bộ nhớ đệm L2 đã thấy được yêu cầu nhưng chưa sẵn sàng để giao dịch nên vẫn giữ tín hiệu Ready ở mức thấp.
- Tại Clock T6, bộ nhớ đệm L2 sẵn sàng nên tích cực tín hiệu Ready.
- Tại Clock T7, bộ nhớ đệm L2 lấy mẫu các tín hiệu dữ liệu (Opcode, Address, Data), tín hiệu Valid và Ready được chuyển về mức thấp, Handshake thành công.

Mạch quản lý quy trình bắt tay (Handshake Handler) là mạch dùng để quan sát và điều khiển hai tín hiệu dùng trong quy trình bắt tay là tín hiệu Valid và Ready, đảm bảo rằng, giao dịch trên các kênh chỉ xảy ra khi dữ liệu truyền đi là có nghĩa và thiết bị nhận đã sẵn sàng để nhận các tín hiệu trên.

Trong thiết kế bài luận, giao dịch giữa hai thiết bị gửi và nhận trên mỗi kênh tín hiệu sẽ được quản lý bởi một bộ quản lý quy trình bắt tay (Handshake Handler) riêng biệt.

3.1.5.1.2 Vùng nhớ

Bộ nhớ (Memory) là nơi lưu trữ dữ liệu của vùng địa chỉ, được cấu thành bởi nhiều Block (hay Cache Line), với mỗi Block lưu trữ một vùng địa chỉ (theo địa chỉ Cache) trong bộ nhớ chính.

Cấu trúc vùng nhớ trong bộ nhớ đệm L2 được thể hiện như sau:

Index	Way	State	Tag	Data	
0x0	Way 0				Block 0
0x0	Way 1				Block 1
0x0	Way 2				Block 2
0x0	Way 3				Block 3
0x1	Way 0				Block 4
0x1	Way 1				Block 5
0x1	Way 2				Block 6
0x1	Way 3				Block 7
		...	...	...	

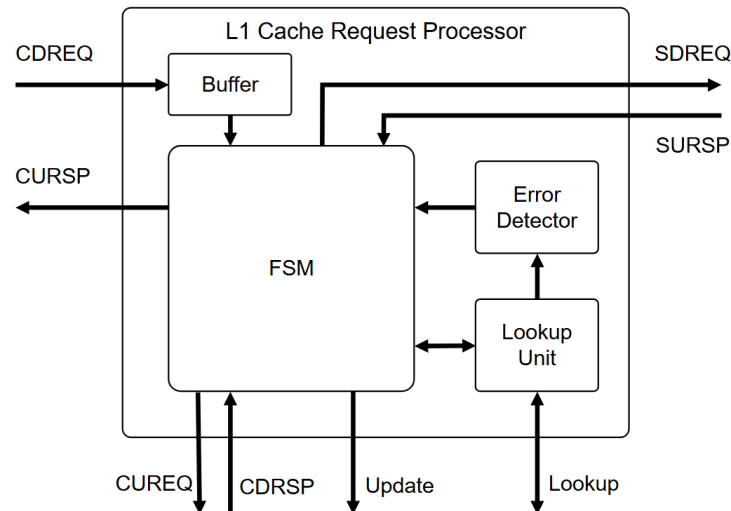
Hình 3-4: Cấu trúc vùng nhớ trong thiết kế

Trong hình trên chúng ta có thể thấy:

- Mỗi Block được cấu thành bởi các trường như trạng thái (State), Tag và dữ liệu (Data).
- Các giá trị Index và Way dùng để xác định địa chỉ có thể được lưu trữ trong Block đó.
- Mỗi giá trị Index sẽ xác định được bốn Block khác nhau (4-Ways). Như vậy, vùng nhớ sẽ có bốn vị trí khả thi để lưu trữ các địa chỉ có cùng trường Index.

Vùng nhớ của thiết kế hỗ trợ phương pháp ánh xạ 4-Ways Set Associative, do đó giá trị tối đa của Index bằng số lượng Block trong bộ nhớ đệm L2 chia cho bốn.

3.1.5.1.3 Bộ xử lý yêu cầu



Hình 3-5: Bộ xử lý yêu cầu từ bộ nhớ đệm L1

Bộ xử lý yêu cầu từ bộ nhớ đệm L1 và từ Snoop Bus đều có các khối như bộ đệm (Buffer), bộ tra cứu bộ nhớ (Lookup Unit), bộ phát hiện lỗi (Error detector) và máy trạng thái (FSM, Finite State Machine). Nhiệm vụ của các khối được miêu tả như sau:

- Bộ đệm (Buffer) dùng để lưu trữ thông tin của yêu cầu như loại yêu cầu (Opcode), địa chỉ (Address) và dữ liệu (Data). Việc này là cần thiết vì sau khi giao dịch trên kênh yêu cầu (xREQ) thành công, các tín hiệu trên sẽ trở về giá trị mặc định, dẫn đến tình trạng thất lạc thông tin của yêu cầu và gây lỗi khi yêu cầu chưa được xử lý xong.
- Bộ tra cứu bộ nhớ (Lookup Unit) là mạch tổ hợp dùng để truy xuất thông tin của địa chỉ được yêu cầu trong bộ nhớ. Kết quả sau tra cứu sẽ được gửi đến bộ phát hiện lỗi (Error Detector) và máy trạng thái (FSM) để xử lý yêu cầu.
- Bộ phát hiện lỗi (Error Detector) dùng để phát hiện các trường hợp không mong muốn dựa trên loại yêu cầu (Opcode), kết quả tra cứu bộ nhớ (Lookup) và trạng thái (State).
- Máy trạng thái (FSM – Finite State Machine) là trung tâm xử lý chính, điều khiển các giai đoạn trong quá trình xử lý yêu cầu.

3.1.6 Bộ xử lý yêu cầu từ bộ nhớ đệm L1

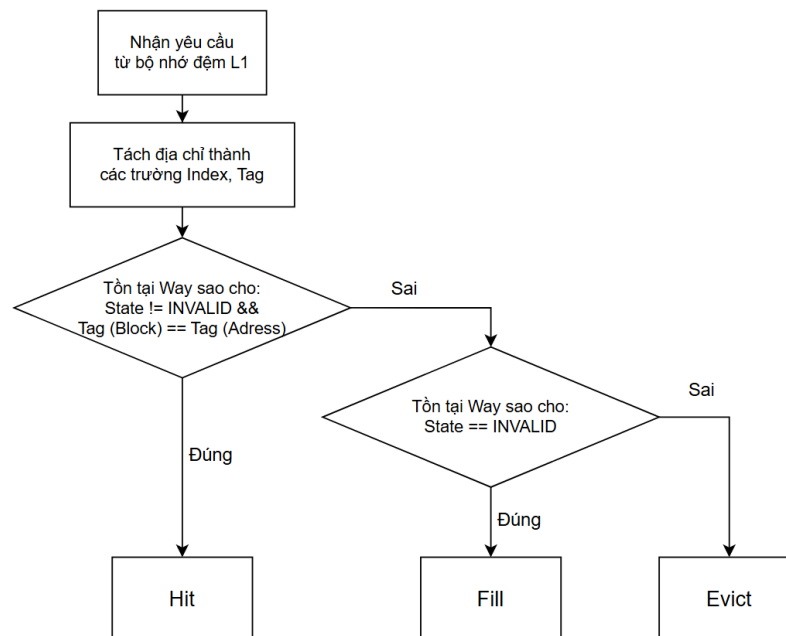
3.1.6.1 Bộ đệm

Bộ đệm (Buffer) trong bộ xử lý yêu cầu từ bộ nhớ đệm L1 dùng để lưu trữ thông tin của yêu cầu trên kênh CDREQ.

Trong quá trình hoạt động, khi bắt tay (Handshake) xảy ra trên kênh CDREQ, bộ đệm (Buffer) sẽ lưu lại thông tin yêu cầu, từ đây, dữ liệu chỉ được giải phóng khi bắt tay trên kênh CURSP được thực hiện.

3.1.6.2 Bộ tra cứu bộ nhớ

Dưới đây là lưu đồ thuật toán của bộ tra cứu bộ nhớ trong bộ xử lý yêu cầu từ bộ nhớ đệm L1:



Hình 3-6: Sơ đồ thuật toán tra cứu bộ nhớ trong bộ xử lý yêu cầu từ bộ nhớ đệm L1

Từ lưu đồ trên, kết quả của tra cứu cho thấy rằng:

- Hit: dữ liệu của địa chỉ được yêu cầu có tồn tại trong bộ nhớ.
- Miss: dữ liệu của địa chỉ được yêu cầu không tồn tại trong bộ nhớ, do đó phải truy xuất dữ liệu từ Snoop Bus. Từ đây tiếp tục được chia thành hai loại.
 - Fill: tồn tại một Way trống (Invalid) để lưu trữ thông tin trả về.
 - Evict: tất cả các Way đều đã được dùng để chứa dữ liệu, do đó phải truy xuất một Way (dựa theo thuật toán thay thế) để lưu trữ thông tin trả về.

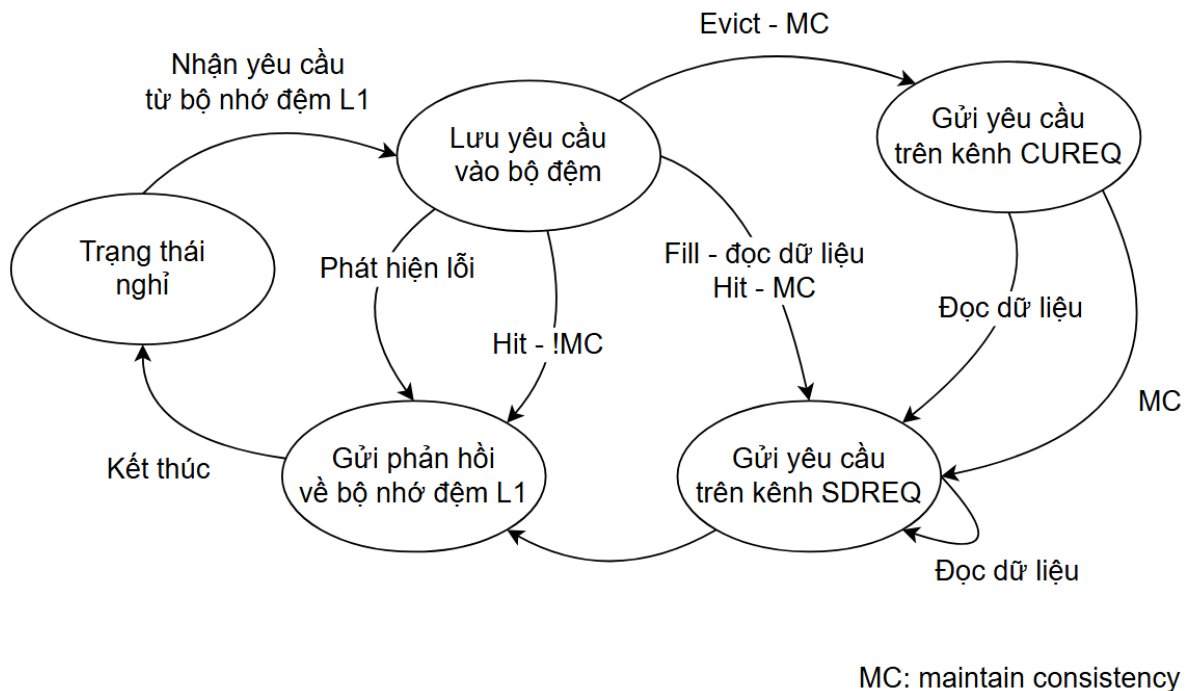
3.1.6.3 Bộ phát hiện lỗi

Danh sách các kịch bản không mong muốn gây ra bởi bộ nhớ đệm L1 xem lại phần 3.2.3.2

Danh sách kịch bản không mong muốn.

3.1.6.4 Máy trạng thái

Hình dưới thể hiện sự thay đổi trạng thái của máy trạng thái trong bộ xử lý yêu cầu từ bộ nhớ đệm L1:



Hình 3-7: Chuyển tiếp trạng thái trong bộ xử lý yêu cầu từ bộ nhớ đệm L1

Quá trình hoạt động của máy trạng thái như sau:

- Máy trạng thái thoát khỏi trạng thái nghỉ khi nhận được yêu cầu từ bộ nhớ đệm L1.
- Dựa vào loại yêu cầu, kết quả tra cứu bộ nhớ và trạng thái của Block mà có thể chuyển sang trạng thái phù hợp để hoàn thành yêu cầu cũng như đảm bảo tính nhất quán trên toàn hệ thống (maintain consistency).
- Nếu có lỗi xảy ra, mạch sẽ được chuyển đến trạng phản hồi lại cho bộ nhớ đệm L1 ngay mà không thực hiện bất kỳ hành động nào.
- Trở về trạng thái nghỉ sau khi hoàn thành yêu cầu và chờ yêu cầu tiếp theo.

3.1.7 Bộ xử lý yêu cầu từ Snoop Bus

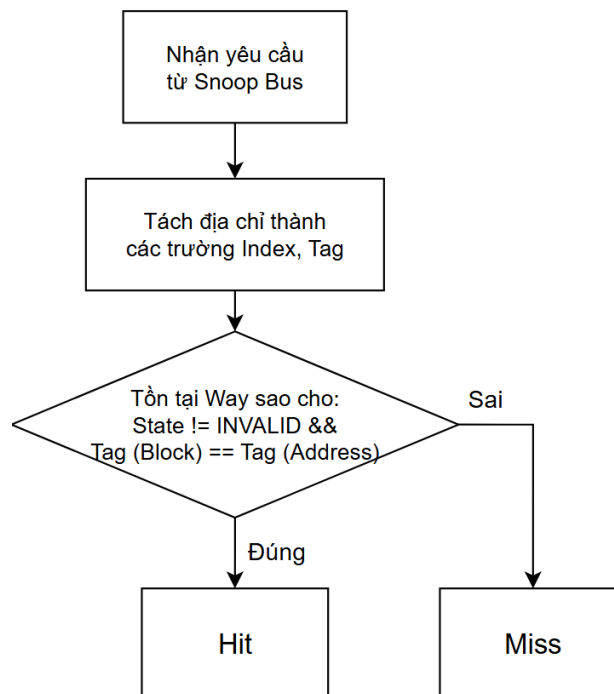
3.1.7.1 Bộ đệm

Bộ đệm (Buffer) trong bộ xử lý yêu cầu từ Snoop Bus dùng để lưu các thông tin của yêu cầu trên kênh SUREQ.

Trong quá trình hoạt động, khi bắt tay (Handshake) xảy ra trên kênh SUREQ, bộ đệm (Buffer) sẽ lưu lại thông tin yêu cầu, từ đây, dữ liệu chỉ được giải phóng khi bắt tay trên kênh SDRSP được thực hiện.

3.1.7.2 Bộ tra cứu bộ nhớ

Dưới đây là lưu đồ thuật toán của bộ tra cứu bộ nhớ trong bộ xử lý yêu cầu từ Snoop Bus:



Hình 3-8: Sơ đồ thuật toán tra cứu bộ nhớ trong bộ xử lý yêu cầu từ Snoop Bus

Từ lưu đồ trên, kết quả của tra cứu cho thấy rằng:

- Hit: dữ liệu của địa chỉ được yêu cầu có tồn tại trong bộ nhớ.
- Miss: dữ liệu của địa chỉ được yêu cầu không tồn tại trong bộ nhớ.

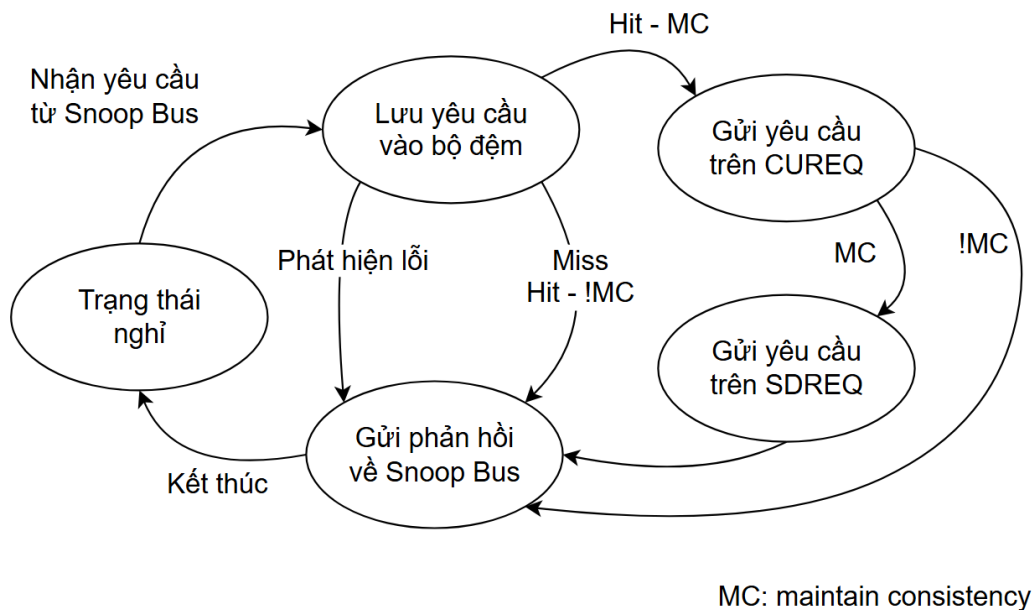
Nếu yêu cầu được gửi từ Snoop Bus và dữ liệu địa chỉ không tồn tại trong bộ nhớ đệm L2, không khiến cho bộ nhớ đệm L2 phải lưu dữ liệu mới, do đó chỉ có kết quả là Miss mà không có Fill hay Evict như yêu cầu từ bộ nhớ đệm L1.

3.1.7.3 Bộ phát hiện lỗi

Danh sách các kịch bản không mong muốn gây ra bởi Snoop Bus xem lại phần 3.2.3.2 [Danh sách kịch bản không mong muốn](#).

3.1.7.4 Máy trạng thái

Hình dưới thể hiện sự thay đổi trạng thái của máy trạng thái trong bộ xử lý yêu cầu từ Snoop Bus:



Hình 3-9: Chuyển tiếp trạng thái trong bộ xử lý yêu cầu từ Snoop Bus

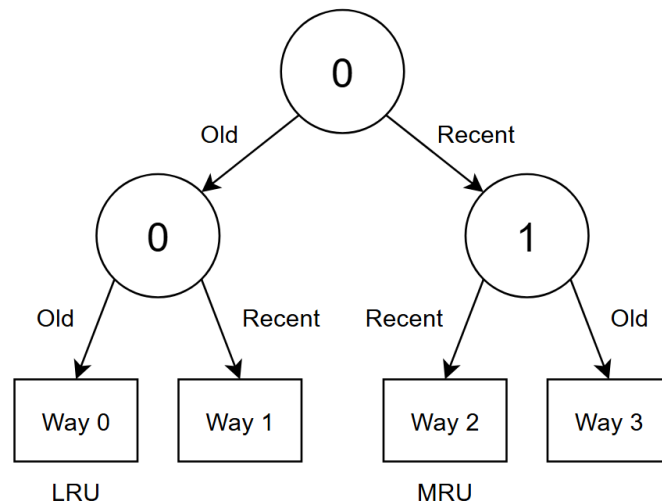
Quá trình hoạt động của máy trạng thái như sau:

- Máy trạng thái thoát khỏi trạng thái nghỉ khi nhận được yêu cầu từ Snoop Bus.
- Dựa vào loại yêu cầu, kết quả tra cứu bộ nhớ và trạng thái của Block mà có thể chuyển sang trạng thái phù hợp để hoàn thành yêu cầu cũng như đảm bảo tính nhất quán trên toàn hệ thống (maintain consistency).
- Nếu có lỗi xảy ra hoặc địa chỉ yêu cầu không tồn tại trong bộ nhớ, mạch sẽ được chuyển đến trạng phản hồi lại cho Snoop Bus ngay mà không thực hiện bất kỳ hành động nào.
- Trở về trạng thái nghỉ sau khi hoàn thành yêu cầu và chờ yêu cầu tiếp theo.

3.1.8 Thuật toán thay thế PLRU

Khác với LRU dùng bộ đếm và so sánh, dẫn đến đòi hỏi nhiều tài nguyên, PLRU dùng cây nhị phân (Binary Tree), tiêu hao ít tài nguyên hơn, để xác định Block nào sẽ bị trục xuất.

Hình dưới miêu tả cây nhị phân trong thuật toán của PLRU:



Hình 3-10: Thuật toán cây nhị phân trong PLRU

Thuật toán của PLRU như sau:

- Giá trị của mỗi nút (Node) trong cây nhị phân sẽ trở đến Way được truy cập gần nhất (MRU, most recently used).
- Khi một Way được truy cập, cây nhị phân sẽ được cập nhập, các nút không dẫn đến Way được truy cập sẽ giữ nguyên giá trị.
- Lúc này, chỉ cần đi ngược chiều với các nút sẽ tìm được Way với lần truy cập cuối xa nhất (LRU, least recently used), hay Way bị trục xuất tiếp theo.

Theo hình trên, nếu giá trị của nút bằng 0 thì sẽ trở vào Way với số thứ tự nhỏ hơn và ngược lại.

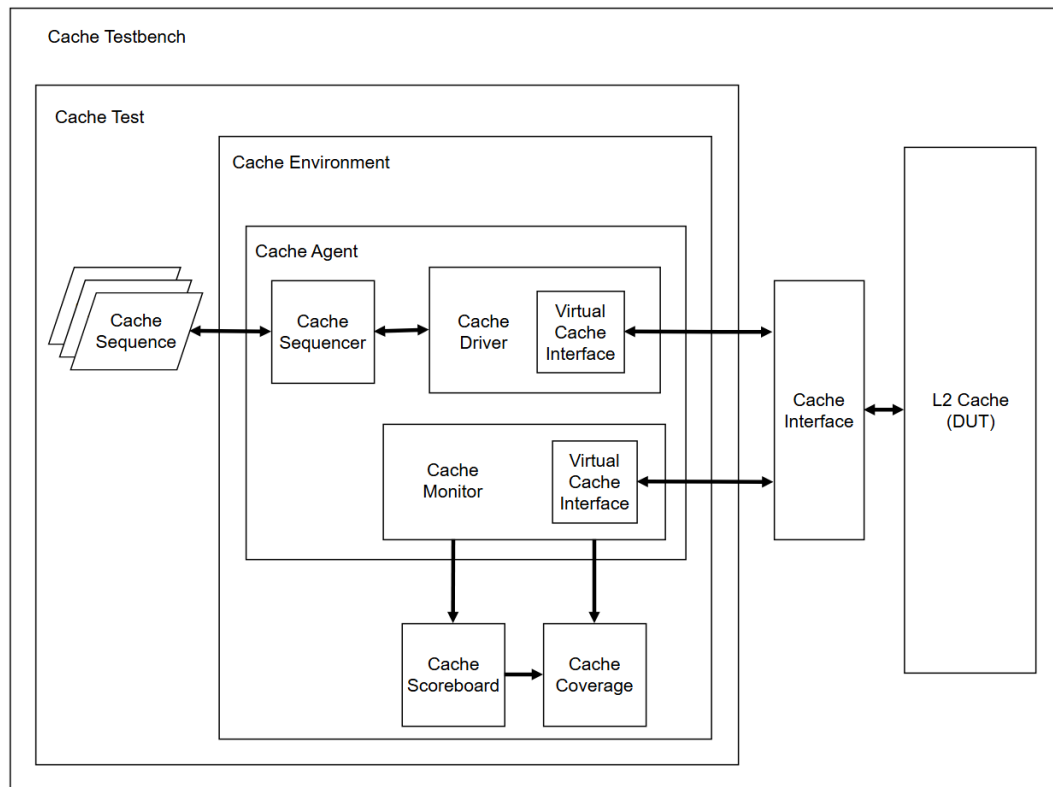
Nếu yêu cầu của Snoop Bus có kết quả tra cứu bộ nhớ là Miss thì giá trị các nút của cây nhị phân sẽ không được cập nhập.

3.2 Kiểm định chức năng của thiết kế

3.2.1 Môi trường UVM

3.2.1.1 Topology

Cấu trúc và phân cấp của môi trường trường UVM được sử dụng trong bài như sau:



Hình 3-11: Môi trường UVM dùng trong bài luận

Trong hình trên chúng ta có thể thấy môi trường UVM trên gồm ba phần chính là mạch được kiểm thử (DUT), Interface và môi trường UVM.

Trong phạm vi bài luận này, mạch được kiểm thử là bộ nhớ đệm L2 hỗ trợ MESI protocol và được mở rộng để đảm bảo tính nhất quán cho bộ nhớ đệm L1, môi trường UVM dùng để tạo các tín hiệu kiểm thử và phân tích và đánh giá kết quả, và cuối cùng là Interface dùng để truyền các tín hiệu kiểm thử từ môi trường UVM đến mạch được kiểm tra.

Interface ảo (Virtual Interface) giúp các thành phần của UVM có thể giao tiếp mà không cần phải ánh xạ trực tiếp đến module Interface trên Testbench, đồng thời tăng tính linh hoạt và tái sử dụng trong các môi trường lớn và phức tạp.

3.2.1.2 Monitor

Trong bài luận này, Monitor sẽ liên tục quan sát và bắt các giao dịch trên mỗi kênh tín hiệu (chỉ bắt tín hiệu khi Handshake thực hiện hoàn tất), từ đó sẽ gửi đến Scoreboard để kiểm tra tính đúng đắn về hành vi của thiết kế. Sau mỗi yêu cầu được hoàn tất, Monitor sẽ đóng gói tất cả các thông tin được giao dịch các kênh có liên quan và gửi cho Coverage để kiểm tra tính bao quát, toàn diện của các kịch bản kiểm thử.

```

62 task `THIS_CLASS::run_phase(uvm_phase phase);
63   fork
64     // packaged into scenario
65     coll_ll_txn();
66     coll_snp_txn();
67
68     // collect data on each channel
69     coll_cdreq_xfr();
70     coll_cursp_xfr();
71     coll_cureq_xfr();
72     coll_cdrsp_xfr();
73     coll_sdreq_xfr();
74     coll_sursp_xfr();
75     coll_sureq_xfr();
76     coll_sdrsp_xfr();
77   join_none
78 endtask: run_phase

```

Hình 3-12: Khái quát hoạt động của Monitor trong bài luận bằng SystemVerilog

Hình trên miêu tả quá trình hoạt động của Monitor bằng ngôn ngữ SystemVerilog, chúng ta có thể thấy quá trình bắt các tín hiệu trên mỗi kênh (từ dòng 69 đến 76) và đóng gói thành kịch bản hoàn chỉnh (dòng 65 và 66) được thực hiện cùng lúc, song song với nhau.

```

291 UVM_INFO ../lib/agents/cache_mon.svh(215) @ 90: uvm_test_top.env.agt.mon [MON] Type_xfr=CDREQ_XFR cdreq_op=CDREQ_RD
    cdreq_addr=0x102 cdreq_data=0x0
292 UVM_INFO ../lib/agents/cache_mon.svh(305) @ 110: uvm_test_top.env.agt.mon [MON] Type_xfr=SDREQ_XFR sdreq_op=SDREQ_RD
    sdreq_addr=0x102 sdreq_data=0x0
293 UVM_INFO ../lib/agents/cache_mon.svh(333) @ 150: uvm_test_top.env.agt.mon [MON] Type_xfr=SURSP_XFR sursp_rsp=SURSP_F
    ETCH sursp_data=0x1102e5d1a8a5750a9f61401d28cd3a7f9b47f9f66b40e593713dfc1f780290577dfe6362b365c7773010c8b3656fee4c3f5
    3dca9553880f1abe65338b220d607
294 UVM_INFO ../lib/agents/cache_mon.svh(235) @ 210: uvm_test_top.env.agt.mon [MON] Type_xfr=CURSP_XFR cursp_rsp=CURSP_O
    KAY cursp_data=0x1102e5d1a8a5750a9f61401d28cd3a7f9b47f9f66b40e593713dfc1f780290577dfe6362b365c7773010c8b3656fee4c3f53
    dca9553880f1abe65338b220d607
295 UVM_INFO ../lib/agents/cache_mon.svh(141) @ 210: uvm_test_top.env.agt.mon [MON] L1 transaction: Type_xfr=L1_TXN Type
    =L1_REQ Lookup=FILL_INV_BLK ST=INVALID IDX=0x2 WAY=0x0 cdreq_op=CDREQ_RD cdreq_addr=0x102 cdreq_data=0x0 sdreq
    _op=SDREQ_RD sdreq_addr=0x102 sdreq_data=0x0 sursp_rsp=SURSP_FETCH sursp_data=0x1102e5d1a8a5750a9f61401d28cd3a7f9b
    47f9f66b40e593713dfc1f780290577dfe6362b365c7773010c8b3656fee4c3f53dca9553880f1abe65338b220d607 cursp_rsp=CURSP_OKAY
    cursp_data=0x1102e5d1a8a5750a9f61401d28cd3a7f9b47f9f66b40e593713dfc1f780290577dfe6362b365c7773010c8b3656fee4c3f53dca95
    53880f1abe65338b220d607

```

Hình 3-13: Monitor in ra Log file trong khi hoạt động

Trong hình trên ta có thể thấy Monitor in ra màn hình các dữ liệu mà nó bắt được trên mỗi kênh như loại yêu cầu, địa chỉ, dữ liệu đi kèm (từ dòng 291 đến 294). Sau mỗi yêu cầu hoàn tất Monitor sẽ đóng gói tất cả thành một kịch bản hoàn chỉnh (dòng 295).

3.2.1.3 Scoreboard

3.2.1.3.1 Mô hình tham chiếu

[13] Mô hình tham chiếu (Reference Model) sẽ mô phỏng lại hành vi của thiết kế dựa theo đặc tính kỹ thuật (Specification), nhưng được viết bằng ngôn ngữ cấp cao hơn để thuận tiện trong việc mô hình hóa và tái sử dụng. Trong bài luận này, mô hình tham chiếu được thực hiện như sau:

```

8  typedef struct packed {
9      data_t data;
10     tag_t tag;
11     st_e state;
12 } block_s;
13
14 block_s mem[('VIP_NUM_BLK/'VIP_NUM_WAY)]['VIP_NUM_WAY];

```

Hình 3-14: Mô hình hóa cấu trúc vùng nhớ bằng SystemVerilog

Cấu trúc của Block được mô phỏng lại bằng kiểu dữ liệu Struct với các trường như trạng thái (State), Tag và dữ liệu (Data). Bộ nhớ là một mảng hai chiều với kiểu dữ liệu là Struct của Block.

Các kiểu dữ liệu của các trường trên được định nghĩa như sau:

```

6  typedef reg ['VIP_BLK_WIDTH-1:0] data_t;
7
8  typedef reg ['VIP_TAG_WIDTH-1:0] tag_t;
9
10 typedef enum reg [2:0] {
11     INVALID = 3'b000,
12     EXCLUSIVE = 3'b001,
13     MODIFIED = 3'b011,
14     MIGRATED = 3'b111,
15     SHARED = 3'b101
16 } st_e;

```

Hình 3-15: Kiểu dữ liệu của các trường trong Block bằng SystemVerilog

Trong quá trình mô phỏng, bộ nhớ trên sẽ được cấu hình và truy xuất thông qua các hàm giao tiếp (hay API, Application Programming Interface).

```

30 extern virtual function idx_t get_idx (address_t addr);
31 extern virtual function way_t get_way (address_t addr, idx_t idx, output lookup_e lookup);
32 extern virtual function st_e get_state (idx_t idx, way_t way);
33 extern virtual function tag_t get_tag (idx_t idx, way_t way);
34 extern virtual function data_t get_data (idx_t idx, way_t way);
35 extern virtual function way_t get_evict_way (idx_t idx);
36
37 extern virtual function void set_state (idx_t idx, way_t way, st_e state);
38 extern virtual function void set_tag (idx_t idx, way_t way, tag_t tag);
39 extern virtual function void set_data (idx_t idx, way_t way, data_t data);

```

Hình 3-16: Các lệnh API giao tiếp với mô hình tham chiếu

Dưới đây là quá trình mà Scoreboard kiểm tra hành vi của thiết kế trong quá trình mô phỏng:

```

296     else if(cdreq_op_req_bf == CDREQ_WB) begin
297         if(cdreq_lookup == HIT) begin
298             if(cdreq_st_prev inside {MIGRATED}) begin
299                 m_cache.set_state(cdreq_idx_bf, cdreq_way_bf, MODIFIED);
300                 m_cache.set_data(cdreq_idx_bf, cdreq_way_bf, t.cdreq_data);
301                 wait_nxt_ll_xfr(t);
302                 if(!((t.Type_xfr == CURSP_XFR) && (t.cursp_rsp == CURSP_OKAY) && (t.cursp_data == '0)))
303                     `SB_ERROR("CDREQ", $sformatf("expect:[CURSP_XFR]   cursp_rsp=CURSP_OKAY   cursp_data=0x0 --- actually:%s", t.convert2string()))
304             end
305             else if(cdreq_st_prev inside {INVALID, EXCLUSIVE, SHARED, MODIFIED}) begin
306                 ll_corner_case_handler(t);
307             end
308             else begin
309                 `uvm_fatal(m_msg_name, $sformatf("invalid ST=%s", cdreq_st_prev.name()))
310             end
311         end
312         else if(cdreq_lookup inside {FILL_INV_BLK, EVICT_BLK}) begin
313             ll_corner_case_handler(t);
314         end
315         else begin
316             `uvm_fatal(m_msg_name, $sformatf("invalid LOOKUP=%s", cdreq_lookup.name()))
317         end
318     end
319     else begin
320         `uvm_fatal(m_msg_name, $sformatf("invalid OP=%s", cdreq_op_req_bf.name()))
321     end

```

Hình 3-17: Ví dụ quá trình kiểm tra hành vi của Scoreboard

Theo hình trên chúng ta có thể thấy khi yêu cầu nhận được là Write Back được gửi từ bộ nhớ đệm L1, mô hình tham chiếu sẽ thực hiện như sau:

- Cấu hình trạng thái của vùng địa chỉ yêu cầu thành Modified.
- Lưu dữ liệu vào vùng địa chỉ được yêu cầu.
- Theo dõi các thông tin phản hồi được gửi trên kênh CURSP, nếu không đúng như mong đợi thì thông báo lỗi cho người dùng..
- Các trường hợp như trạng thái ban đầu của vùng địa chỉ không phải là Migrated hay vùng địa chỉ được yêu cầu không tồn tại trong bộ nhớ L2 (hay kết quả tra cứu bộ nhớ không phải là Hit) sẽ được đánh dấu là trường hợp không hợp lệ và thông báo cho người dùng.

3.2.1.3.2 Đối chiếu thông tin giữa mô hình tham chiếu và thiết kế

Trong bài luận này, ngoài kiểm tra hành vi của các tín hiệu trên Interface, Scoreboard còn hỗ trợ thêm kỹ thuật có thể truy cập trực tiếp vào thanh ghi của thiết kế mà không cần thông qua Interface (hay được gọi là Read Backdoor).

Sau khi thực hiện xong mỗi yêu cầu, Scoreboard sẽ đọc toàn bộ thông tin của Block vừa được truy cập bao gồm State, Tag và Data, sau đó so sánh các giá trị trên với các trường của Block tương ứng trong mô hình tham chiếu và thông báo cho người dùng nếu có khác nhau xảy ra.

Hình bên dưới là kết quả báo cáo sau khi đối chiếu hoàn tất:

```

341 UVM_INFO ../lib/units/cache_sb_base.svh(195) @ 490: uvm_test_top.env.sb [SB] no mismatch between
    RTL and MODEL at: IDX=2 WAY=1
342 UVM_INFO ../lib/units/cache_sb.svh(343) @ 490: uvm_test_top.env.sb [PASS_CDREQ] Report:
343 UVM_INFO ../lib/units/cache_sb.svh(347) @ 490: uvm_test_top.env.sb [SB] ST=SHARED <-- INVALID
344 UVM_INFO ../lib/units/cache_sb.svh(351) @ 490: uvm_test_top.env.sb [SB] TAG=0x2 <-- 0x0
345 UVM_INFO ../lib/units/cache_sb.svh(355) @ 490: uvm_test_top.env.sb [SB] DATA=0x79ab283d49e0d5760c
    ea10d37fb0892dfa19a665d78412a3222634618a52fe81c7fc37b580d80d54976aeadd11919e093f37131bfaf6ed76d0ebd
    ede13437a039 <-- 0x0

```

Hình 3-18: Kết quả đối chiếu mô hình tham chiếu và thiết kế

Chúng ta thấy sau khi thực hiện yêu cầu hoàn tất, trạng thái của Block được truy cập đã chuyển từ Invalid sang Shared và lưu trữ giá trị mới trên trường Tag và Data.

Ngoài đối chiếu thông tin của Block được truy cập sau mỗi lần hoàn thành một yêu cầu. Khi tất cả yêu cầu của bài Test đã được thực hiện, Scoreboard sẽ đối chiếu thông tin toàn bộ bộ nhớ (tất cả Block trong bộ nhớ đệm) để đảm bảo tính chính xác giữa mô hình tham chiếu và thiết kế.

3.2.1.3.3 Tổng kết kiểm thử

Khi không còn yêu cầu nào cần gửi, Scoreboard sẽ tổng kết tất cả các yêu cầu đã được gửi trong toàn bộ bài Test. Theo hình bên dưới chúng ta có thể thấy số yêu cầu được gửi, số yêu cầu hợp lệ, yêu cầu không mong muốn, và số yêu cầu với các trường hợp tra cứu bộ nhớ khác nhau.

```

_c] Complete test
[SB] no mismatch between RTL and MODEL at all blocks
[SB] ---- SUMMARY CDREQ ----
[SB] number CDREQ functional case: 536
[SB] number CDREQ corner case: 418
[SB] number CDREQ hit: 285
[SB] number CDREQ fill: 239
[SB] number CDREQ evict: 12
[SB] hit rate CDREQ: 53.000000
[SB] -----
[SB] ---- SUMMARY SUREQ ----
[SB] number SUREQ functional case: 931
[SB] number SUREQ corner case: 115
[SB] number SUREQ hit: 500
[SB] number SUREQ miss: 546
[SB] hit rate SUREQ: 53.000000
[SB] -----

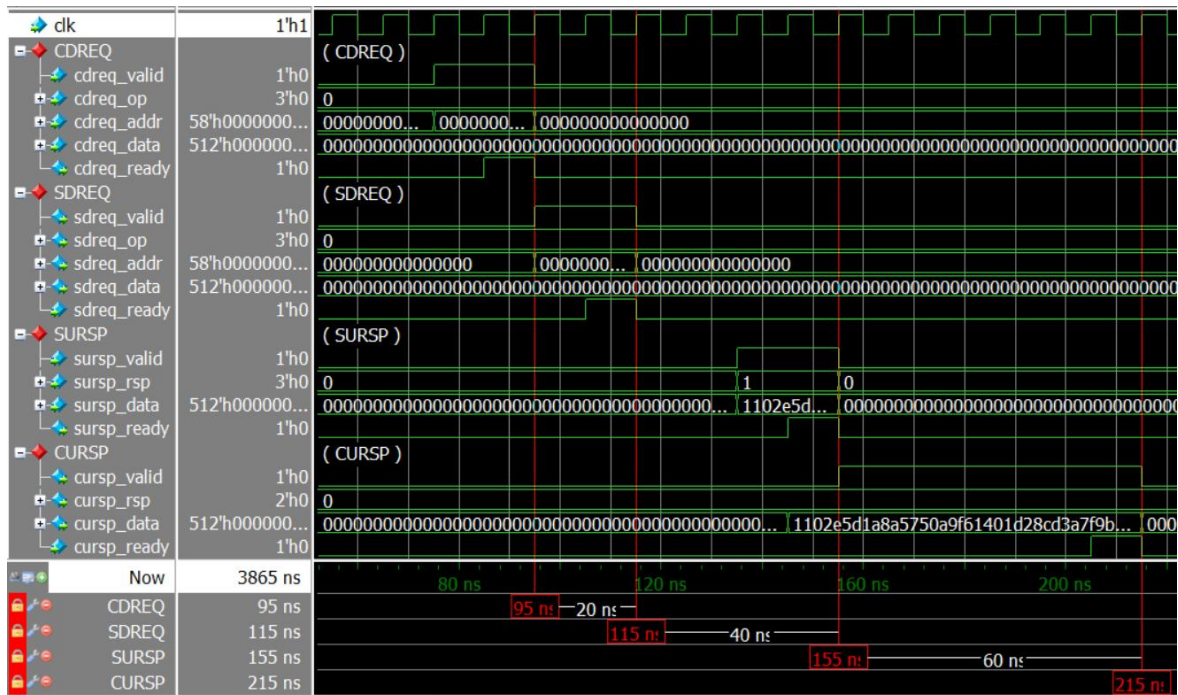
```

Hình 3-19: Tổng kết quá trình kiểm thử

3.2.2 Dạng sóng tín hiệu

Để chứng minh thiết kế hoạt động đúng với mong đợi cần phải có các công cụ để quan sát hành vi của thiết kế trong khi thực hiện các chức năng. Trong đó, dạng sóng (waveform) chứa các hành vi, giá trị của tín hiệu thiết kế theo thời gian.

Dưới đây là một ví dụ của dạng sóng hoạt động của thiết kế:



Hình 3-20: Dạng sóng tín hiệu của thiết kế khi xử lý yêu cầu

Ở dạng sóng trên chúng ta có thể thấy:

- Tín hiệu Clock giúp đồng bộ các thành phần trong thiết kế.
- Các tín hiệu của bốn kênh CDREQ, CURSP, SDREQ và SURSP bao gồm tên tín hiệu, giá trị tại thời điểm nhất định và dạng sóng theo thời gian.
- Mỗi kênh tín hiệu sẽ có hai tín hiệu thực hiện cơ chế bắt tay (handshake), khi cả hai tín hiệu được tích cực và trở về giá trị không sau đó một xung Clock, ta nói giao dịch giữa hai thiết bị trên kênh tương ứng đã thành công.
- Phía dưới dạng sóng thể hiện thời gian mô phỏng.
- Trên từng thời điểm của dạng sóng chúng ta có thể thấy các con trỏ (cursor) nằm trên các thời điểm 95ns, 115ns, 155ns, 215ns lần lượt là các thời điểm cơ chế bắt tay hoàn tất của các kênh CDREQ, SDREQ, SURSP, CURSP. Tên của các con trỏ nằm ở góc dưới, bên trái.

3.2.3 Kế hoạch kiểm thử

3.2.3.1 Danh sách kịch bản có thể xảy ra

Để kiểm tra thiết kế đã hoạt động đúng như mong đợi, bài luận này sẽ tạo ra tất cả các kịch bản có thể xảy ra, bao gồm các kịch bản hợp lệ và không mong muốn, sau đó đánh giá tín hiệu trả về cũng như dữ liệu được lưu trữ trong vùng nhớ của thiết kế.

Danh sách các kịch bản có thể xảy ra được thể hiện ở dưới:

Bảng 3-6: Danh sách các kịch bản hợp lệ của thiết kế

Channel	Opcode	Lookup	State				
			Invalid	Exclusive	Shared	Modified	Migrated
CDREQ	RD	Hit	-	Y	Y	Y	X
		Fill	Y	-	-	-	-
		Evict	-	Y	Y	Y	Y
	RFO	Hit	-	Y	Y	Y	N
		Fill	Y	-	-	-	-
		Evict	-	Y	Y	Y	Y
	MD	Hit	-	Y	Y	Y	Y
		Fill	N	-	-	-	-
		Evict	-	N	N	N	N
	WB	Hit	-	N	N	N	Y
		Fill	N	-	-	-	-
		Evict	-	N	N	N	N
SUREQ	RD	Hit	-	Y	Y	Y	Y
		Miss	Y	-	-	-	-
	RFO	Hit	-	Y	Y	Y	Y
		Miss	Y	-	-	-	-
	INV	Hit	-	N	Y	N	N
		Miss	Y	-	-	-	-

Ký hiệu của bảng trên như sau:

- “ Y “: Yes. Kịch bản hợp lệ, có thể xảy ra.
- “ N “: No. Kịch bản không mong muốn, lỗi do các thiết bị gửi yêu cầu gây nên.
- “ – “: Lỗi logic diễn ra trong bộ nhớ đệm L2.

Trong hình trên, các kịch bản sẽ được cấu hình bằng thiết bị gửi yêu cầu, loại lệnh yêu cầu, kết quả tra cứu bộ nhớ, trạng thái hiện tại của dữ liệu.

3.2.3.2 Giải thích kịch bản không mong muốn

Danh sách các kịch bản không mong muốn được thể hiện ở dưới:

Bảng 3-7: Danh sách các kịch bản không mong muốn trong thiết kế

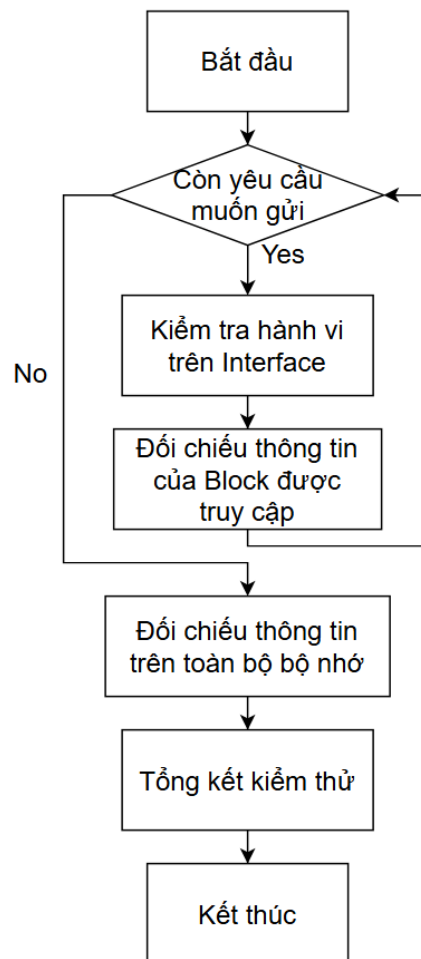
Channel	Opcode	Lookup	State	Miêu tả
CDREQ	RD / RFO	Hit	Migrated	Bộ nhớ đệm L1 phải nhận thức được rằng nó đang lưu trữ dữ liệu mới nhất, do đó không nên gửi RD (Read) hay RFO (Read for ownership).
	MD	Fill / Evict	X	Lệnh MD (Mark Dirty) được gửi, đồng nghĩa với địa chỉ yêu cầu có tồn tại trong L1. Do vậy nếu địa chỉ đó không tồn tại trong L2 sẽ vi phạm tính bao hàm (Inclusivity Property).
	WB	Hit	Exclusive/ Shared/ Modified	Khi bộ nhớ đệm L1 gửi Write Back xuống L2, nó phải thông báo rằng vùng địa này đã được chỉnh sửa bằng lệnh MD (Mark Dirty) trước.
	WB	Fill / Evict	X	Lệnh WB được gửi khi bộ nhớ đệm L1 muốn Write Back dữ liệu xuống L2. Trong trường hợp này, vùng địa chỉ yêu cầu không tồn tại trong L2 sẽ vi phạm tính bao hàm (Inclusivity Property).
SUREQ	INV	Hit	Exclusive/ Modified/ Migrated	Trong các trạng thái trên, không có bộ nhớ đệm L2 nào chứa bản sao của vùng địa chỉ được yêu cầu, nên lệnh Invalidate không thể được tạo ra.

Ký hiệu của bản trên như sau:

- “ X “: Do not care. Ý thể hiện rằng có thể là bất kỳ trạng thái nào.

3.2.3.3 Trình tự kiểm tra của Scoreboard

Dưới đây là hoạt động của Scoreboard trong quá trình mô phỏng.



Hình 3-21: Quy trình hoạt động của Scoreboard của thiết kế

Dựa vào hình trên chúng ta có thể thấy:

- Với mỗi yêu cầu được gửi, Scoreboard sẽ đối chiếu tín hiệu trên các kênh với mô hình tham chiếu để kiểm tra hành vi của thiết kế.
- Sau khi yêu cầu được hoàn thành, Scoreboard sẽ đối chiếu thông tin Block được truy cập của thiết kế (RTL) với mô hình tham chiếu để kiểm tra thông tin được lưu trữ trong bộ nhớ
- Khi các yêu cầu của bài Test đã được gửi, Scoreboard sẽ đối chiếu toàn bộ bộ nhớ của thiết kế với mô hình tham chiếu để kiểm tra lần cuối.
- Cuối cùng Scoreboard thống kê các số liệu của bài Test.

3.2.3.4 Coverage

3.2.3.4.1 Coverage cho các kịch bản hợp lệ

Covergroup sẽ có các coverpoint sẽ bao phủ các thông số sau:

Bảng 3-8: Coverage cho các kịch bản hợp lệ

Thông số được bao phủ	Miêu tả
Các yêu cầu ban đầu	Bao phủ hết các encode của tín hiệu Opcode trên hai kênh CDREQ và SDREQ
Kết quả tra cứu bộ nhớ	Bao phủ hết các kết quả tra cứu như Hit, Fill, Evict, Snoop Miss
Trạng thái Block	Bao phủ hết các trạng thái của MESI và một trạng thái mở rộng của bài luận là Migrated
Các yêu cầu phát sinh	Bao phủ hết các encode của tín hiệu Opcode trên hai kênh CUREQ và SDREQ
Kết quả phản hồi từ kênh SURSP	Bao phủ hết các encode của tín hiệu Response trên kênh SURSP

Ngoài ra còn hỗ trợ thêm các Cross Coverpoint để bao quát hết các tổ hợp có thể xảy ra:

- Yêu cầu từ SDREQ và phản hồi từ SURSP:

```
CROSS_SDREQ_OP_X_SURSP_RSP_COV: cross CP_SDREQ_OP_COV, CP_SURSP_RSP_COV
```

- Yêu cầu từ SDREQ, kết quả tra cứu bộ nhớ và trạng thái dữ liệu:

```
CROSS_CDREQ_OP_X_LOOKUP_X_STATE: cross CP_CDREQ_OP_COV, CP_LOOKUP_COV, CP_STATE_COV
```

3.2.3.4.2 Coverage các kịch bản không mong muốn

Các Coverpoint sẽ đảm bảo bao phủ hết các kịch bản không mong muốn đã được liệt kê trong bài luận (xem lại phần 3.2.3.2 [Danh sách kịch bản không mong muốn](#).)

Dưới đây là ví dụ Coverpoint cho kịch bản bộ nhớ đệm L1 gửi thông báo Mark Dirty cho vùng địa chỉ không tồn tại trong bộ nhớ đệm L2 (kịch bản không mong muốn):

```

98  CP_MD_X_MISS_COV: coverpoint m_txn.Lookup iff (m_txn.cdreq_op == CDREQ_MD) {
99      bins          CB_MD_FILL  = {FILL_INV_BLK};
100     bins          CB_MD_EVICT = {EVICT_BLK};
101 }
```

Hình 3-22: Ví dụ Coverpoint cho một kịch bản không hợp lệ

3.2.3.4.3 Đọc kết quả Coverage trên công cụ mô phỏng

Các thông số của Coverage (tính theo %) có thể được theo dõi trên công cụ mô phỏng để hỗ trợ người dùng dễ dàng theo dõi và đánh giá mức độ bao phủ trên thiết kế.

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/cache_units_pkg/cache_cov_cdreq_c		56.1%				
TYPE CG_CDREQ_MESI_COV	cache_cov...	77.5%	100	77.5%		✓
CVP CG_CDREQ_MESI_COV::CP_STATE_COV	cache_cov...	80.0%	100	80.0%		✓
CVP CG_CDREQ_MESI_COV::CP_CDREQ_OP_COV	cache_cov...	75.0%	100	75.0%		✓
CVP CG_CDREQ_MESI_COV::CP_LOOKUP_COV	cache_cov...	100.0%	100	100.0%		✓
CVP CG_CDREQ_MESI_COV::CP_CUREQ_OP_COV	cache_cov...	50.0%	100	50.0%		✓
CVP CG_CDREQ_MESI_COV::CP_CDRESP_RSP_COV	cache_cov...	100.0%	100	100.0%		✓
CVP CG_CDREQ_MESI_COV::CP_SDREQ_OP_COV	cache_cov...	66.6%	100	66.6%		✓
CVP CG_CDREQ_MESI_COV::CP_SURSP_RSP_COV	cache_cov...	100.0%	100	100.0%		✓
CVP CG_CDREQ_MESI_COV::CP_CURSP_RSP_COV	cache_cov...	100.0%	100	100.0%		✓
CROSS CG_CDREQ_MESI_COV::CROSS_SDREQ_OP_X_SURSP_RSP_COV	cache_cov...	75.0%	100	75.0%		✓
CROSS CG_CDREQ_MESI_COV::CROSS_CDREQ_OP_X_LOOKUP_X_STATE	cache_cov...	28.5%	100	28.5%		✓
INST \cache_units_pkg::cache_cov_cdreq_c::CG_CDREQ_MESI_COV		77.5%	100	77.5%		✓
TYPE CG_CDREQ_CORNER_CASE_COV	cache_cov...	56.6%	100	56.6%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_CURSP_RSP_COV	cache_cov...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_L1_FETCH_HIT_MIGRATED_COV	cache_cov...	50.0%	100	50.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_WB_X_HIT_NOT_MIGRATED_COV	cache_cov...	33.3%	100	33.3%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_MD_X_MISS_COV	cache_cov...	50.0%	100	50.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_WB_X_MISS_COV	cache_cov...	50.0%	100	50.0%		✓
INST \cache_units_pkg::cache_cov_cdreq_c::CG_CDREQ_CORNER_CASE_COV		56.6%	100	56.6%		✓
TYPE CG_CDREQ_BLK_COV	cache_cov...	34.1%	100	34.1%		✓
/cache_units_pkg/cache_cov_sureq_c		54.7%				
TYPE CG_SUREQ_MESI_COV	cache_cov...	76.2%	100	76.2%		✓
TYPE CG_SUREQ_CONER_CASE_COV	cache_cov...	33.3%	100	33.3%		✓

Hình 3-23: Theo dõi kết quả Coverage bằng công cụ mô phỏng

3.2.4 Mô phỏng kịch bản hợp lệ

Dưới đây là một vài trường hợp lệ dựa theo chuẩn MESI protocol và các tính năng mở rộng trong bài luận.

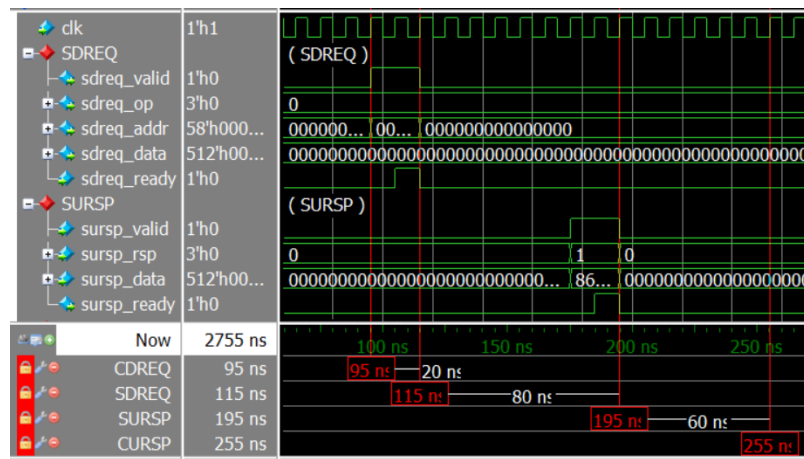
3.2.4.1 Bộ nhớ đệm L1 đọc vùng địa chỉ không tồn tại trong bộ nhớ đệm L2

Chi tiết kịch bản như sau:

- Bộ nhớ đệm L1 gửi yêu cầu đọc dữ liệu.
- Bản sao của địa chỉ yêu cầu không tồn tại trong bộ nhớ đệm L2.
- Không có bộ nhớ đệm nào trong hệ thống có bản sao của địa chỉ yêu cầu.
- Bộ nhớ đệm L2 không cần phải trục xuất Block để lưu dữ liệu mới.

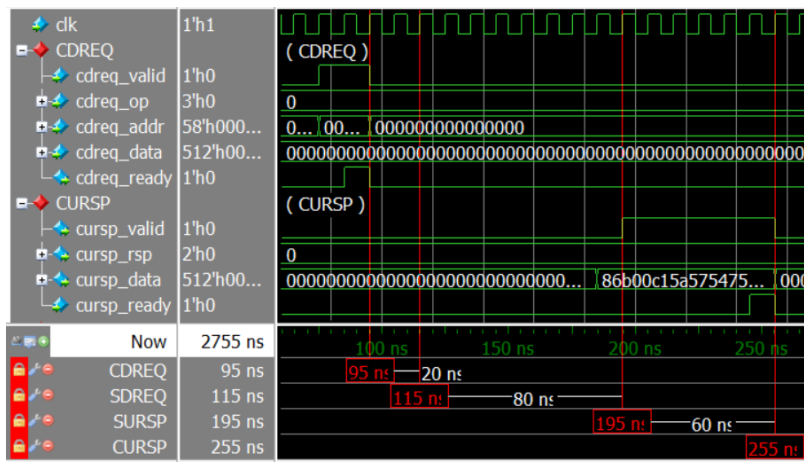
Hành vi của thiết kế được thể hiện như sau:

- Bộ nhớ đệm L2 gửi yêu cầu đọc dữ liệu xuống Snoop Bus.



Hình 3-24: Dạng sóng kênh SDREQ và SURSP của ví dụ

- Sau khi nhận phản hồi từ Snoop Bus, bộ nhớ đệm L2 gửi dữ liệu nhận được cho bộ nhớ đệm L1.



Hình 3-25: Dạng sóng trên kênh CDREQ và CURSP của ví dụ

- Bản sao được tạo trong bộ nhớ và có trạng thái là Exclusive.

```

263 UVM_INFO ../Lib/units/cache_sb.svh(117) @ 90: uvm_test_top.env.sb [SB] ----- CDREQ -----
264 UVM_INFO ../Lib/units/cache_sb.svh(118) @ 90: uvm_test_top.env.sb [REC_CDREQ] Request: OP=CDREQ_RD ADDR=0x102
    DAT=0x0 Lookup=FILL_INV_BLK Access to: IDX=0x2 WAY=0x0 ST=INVALID TAG=0x0 DAT=0x0
265 UVM_INFO ../Lib/units/cache_sb_base.svh(195) @ 260: uvm_test_top.env.sb [SB] no mismatch between RTL and MODEL
    at: IDX=2 WAY=0
266 UVM_INFO ../Lib/units/cache_sb.svh(343) @ 260: uvm_test_top.env.sb [PASS_CDREQ] Report allocation
267 UVM_INFO ../Lib/units/cache_sb.svh(347) @ 260: uvm_test_top.env.sb [SB] ST=EXCLUSIVE <-- INVALID
268 UVM_INFO ../Lib/units/cache_sb.svh(351) @ 260: uvm_test_top.env.sb [SB] TAG=0x1 <-- 0x0
269 UVM_INFO ../Lib/units/cache_sb.svh(355) @ 260: uvm_test_top.env.sb [SB] DATA=0x86b00c15a5754754d4173433e8d0f1bb
    56abc90871ed061ad4cb5c0a7488831f714ac34c1130bcc24805d827c5899d6995b0eecd4de7b479561aa13c57abee9 <-- 0x0
270 UVM_INFO ../Lib/units/cache_sb.svh(365) @ 260: uvm_test_top.env.sb [SB] -----
    
```

Hình 3-26: Kết quả đối chiếu của ví dụ

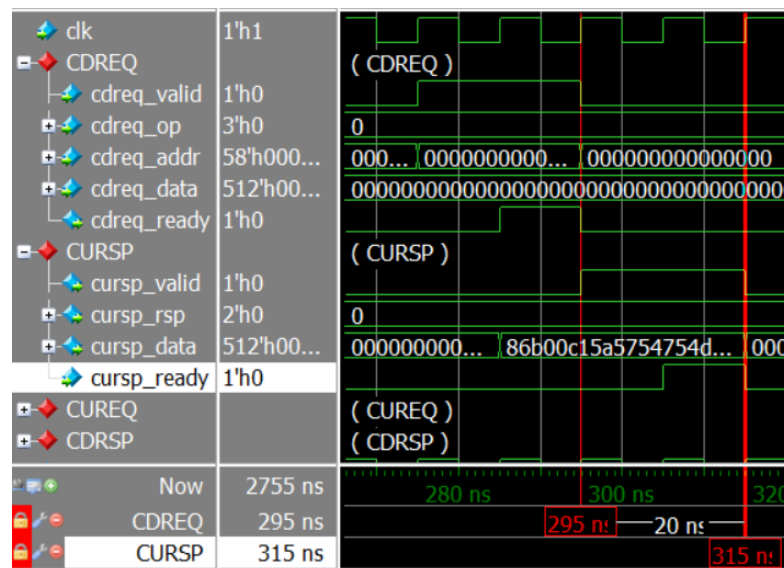
3.2.4.2 Bộ nhớ đệm L1 đọc vùng địa chỉ có tồn tại trong bộ nhớ đệm L2

Chi tiết kịch bản như sau:

- Bộ nhớ đệm L1 gửi yêu cầu đọc dữ liệu.
- Bản sao của địa chỉ yêu cầu có tồn tại trong bộ nhớ đệm L2.
- Trạng thái của dữ liệu là Exclusive.

Hành vi của thiết kế được thể hiện như sau:

- Bộ nhớ đệm L2 gửi dữ liệu được yêu cầu cho bộ nhớ đệm L1.



Hình 3-27: Dạng sóng trên kênh CDREQ và CURSP của ví dụ

- Thông tin của bản sao không thay đổi.

```

272 UVM_INFO ../lib/units/cache_sb.svh(117) @ 290: uvm_test_top.env.sb [SB] ----- CDREQ -----
273 UVM_INFO ../lib/units/cache_sb.svh(118) @ 290: uvm_test_top.env.sb [REC_CDREQ] Request: OP=CDREQ_RD ADDR=0x102
    DAT=0x0 Lookup=HIT Access to: IDX=0x2 WAY=0x0 ST=EXCLUSIVE TAG=0x1 DAT=0x86b00c15a5754754d4173433e8d0f1b
    b56abc90871ed061ad4cb5c0a7488831f714ac34c1130bcc24805d827c5899d6995b0eecd4de7b479561aa13c57abee9
274 UVM_INFO ../lib/units/cache_sb_base.svh(195) @ 320: uvm_test_top.env.sb [SB] no mismatch between RTL and MODEL
    at: IDX=2 WAY=0
275 UVM_INFO ../lib/units/cache_sb.svh(343) @ 320: uvm_test_top.env.sb [PASS_CDREQ] Report allocation
276 UVM_INFO ../lib/units/cache_sb.svh(345) @ 320: uvm_test_top.env.sb [SB] ST=EXCLUSIVE(remain)
277 UVM_INFO ../lib/units/cache_sb.svh(349) @ 320: uvm_test_top.env.sb [SB] TAG=0x1(remain)
278 UVM_INFO ../lib/units/cache_sb.svh(353) @ 320: uvm_test_top.env.sb [SB] DATA=0x86b00c15a5754754d4173433e8d0f1bb
    56abc90871ed061ad4cb5c0a7488831f714ac34c1130bcc24805d827c5899d6995b0eecd4de7b479561aa13c57abee9(remain)
279 UVM_INFO ../lib/units/cache_sb.svh(365) @ 320: uvm_test_top.env.sb [SB] -----
    
```

Hình 3-28: Kết quả đối chiếu của ví dụ

3.2.4.3 Bộ nhớ đệm L2 trực xuất Block để lưu trữ địa chỉ mới

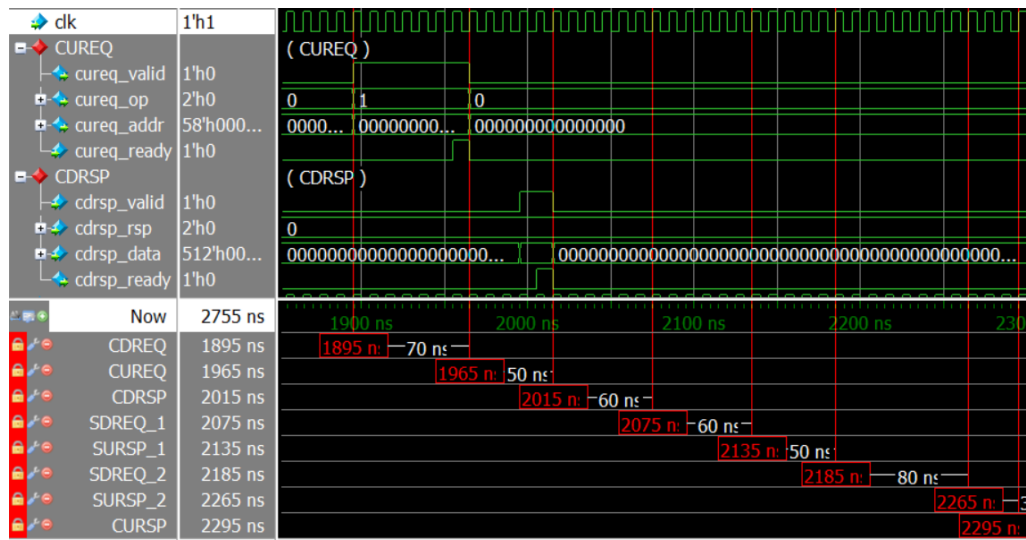
Chi tiết kịch bản như sau:

- Bộ nhớ đệm L1 gửi yêu cầu đọc dữ liệu.
- Bản sao của địa chỉ yêu cầu không tồn tại trong bộ nhớ đệm L2.
- Tồn tại bộ nhớ đệm trong hệ thống có bản sao của địa chỉ yêu cầu.

- Bộ nhớ đệm L2 cần phải trục xuất Block để lưu dữ liệu mới..
- Trạng thái của dữ liệu hiện tại là Migrated

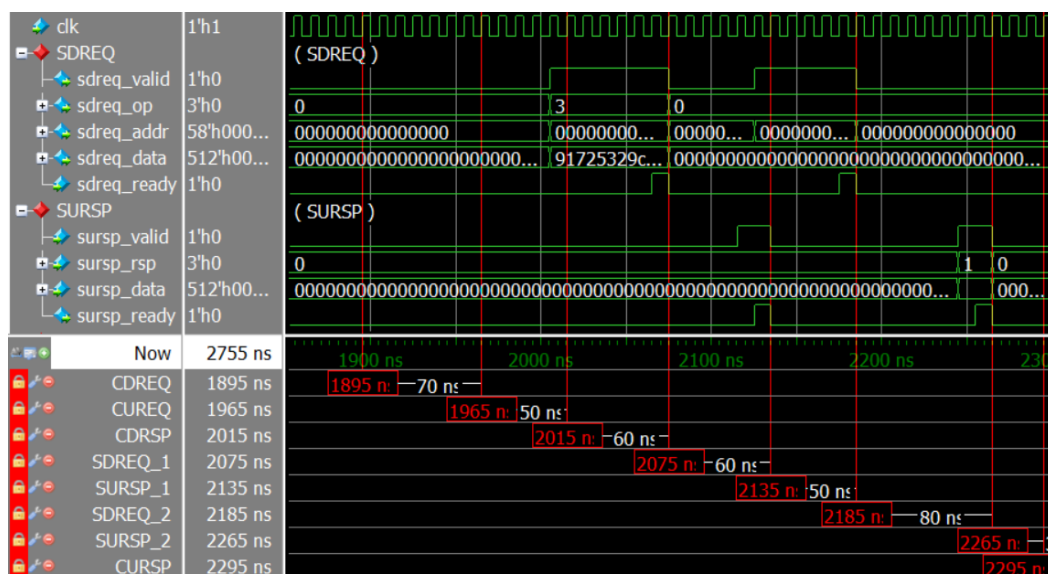
Hành vi của thiết kế được thể hiện như sau:

- Bộ nhớ đệm L2 gửi yêu cầu đọc và xóa bản sao (RFO) cho bộ nhớ đệm L1.



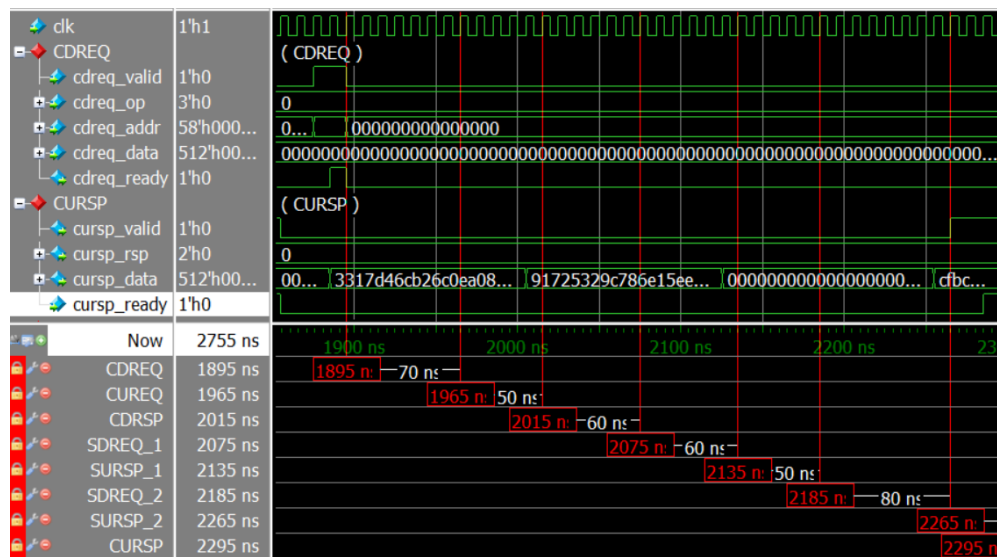
Hình 3-29: Dạng sóng trên kênh CUREQ và CDRSP của ví dụ

- Bộ nhớ đệm L2 gửi Write Back dữ liệu của bản sao bị trục xuất xuống Snoop Bus, sau đó tiếp tục gửi yêu cầu đọc dữ liệu của địa chỉ được yêu cầu.



Hình 3-30: Dạng sóng kênh SDREQ và SURSP của ví dụ

- Sau khi nhận phản hồi từ Snoop Bus, bộ nhớ đệm L2 gửi dữ liệu nhận được cho bộ nhớ đệm L1.



Hình 3-31: Dạng sóng trên kênh CDREQ và CURSP của ví dụ

- Bản sao được tạo trong bộ nhớ đệm L2

```

371 UVM_INFO ../lib/units/cache_sb.svh(117) @ 1890: uvm_test_top.env.sb [SB] ----- CDREQ -----
372 UVM_INFO ../lib/units/cache_sb.svh(118) @ 1890: uvm_test_top.env.sb [REC_CDREQ] Request: OP=CDREQ_RD ADDR=0x50
3 DAT=0x0 Lookup=EVICT_BLK Access to: IDX=0x3 WAY=0x0 ST=MIGRATED TAG=0x1 DAT=0x3317d46cb26c0ea0834167638
129fe6f0aebf168a6fc7befe1a0d0a1fb4acc15dbc8f41cba6be78247bcf760ae3183f9a4ebfcfabf2852648a6fcdac3d731e9c
373 UVM_INFO ../lib/units/cache_sb_base.svh(195) @ 2300: uvm_test_top.env.sb [SB] no mismatch between RTL and MODEL
at: IDX=3 WAY=0
374 UVM_INFO ../lib/units/cache_sb.svh(343) @ 2300: uvm_test_top.env.sb [PASS_CDREQ] Report allocation
375 UVM_INFO ../lib/units/cache_sb.svh(347) @ 2300: uvm_test_top.env.sb [SB] ST=EXCLUSIVE <-- MIGRATED
376 UVM_INFO ../lib/units/cache_sb.svh(351) @ 2300: uvm_test_top.env.sb [SB] TAG=0x5 <-- 0x1
377 UVM_INFO ../lib/units/cache_sb.svh(355) @ 2300: uvm_test_top.env.sb [SB] DATA=0xcfbcdcf6918b3177ea094b93ab8d43e
a739eb7dcdd489fad3c57d6c7065b73126f3e1820e422bd52193e9ea28dde572105fd224b2354c25446e3c2a986de16a <-- 0x3317d46c
b26c0ea0834167638129fe6f0aebf168a6fc7befe1a0d0a1fb4acc15dbc8f41cba6be78247bcf760ae3183f9a4ebfcfabf2852648a6fcdac
3d731e9c
    
```

Hình 3-32: Kết quả đối chiếu của ví dụ

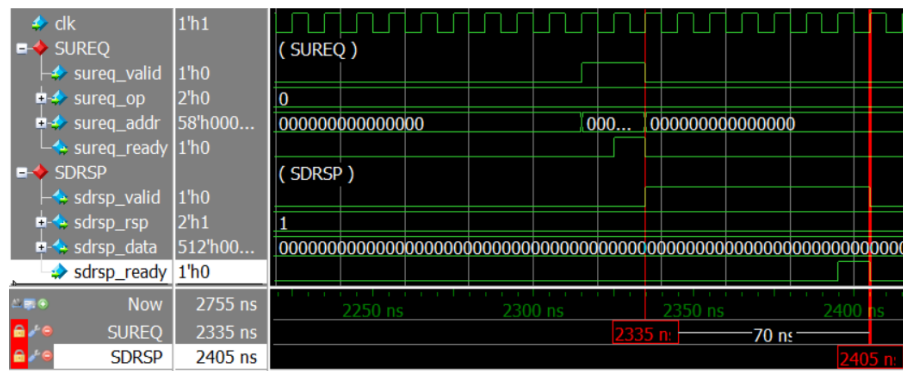
3.2.4.4 Snoop Bus đọc vùng địa chỉ không tồn tại trong bộ nhớ đệm L2

Chi tiết kịch bản như sau:

- Snoop Bus gửi yêu cầu đọc dữ liệu.
- Bản sao của địa chỉ yêu cầu không tồn tại trong bộ nhớ đệm L2.

Hành vi của thiết kế được thể hiện như sau:

- Bộ nhớ đệm L2 gửi phản hồi thể hiện không chứa bản sao (Invalid) và không kèm theo dữ liệu.



Hình 3-33: Dạng sóng trên kênh SUREQ và SDRSP của ví dụ

- Không có sự thay đổi trong bộ nhớ đệm L2.

```

380 UVM_INFO ../lib/units/cache_sb.svh(432) @ 2330: uvm_test_top.env.sb [SB] ----- SUREQ -----
381 UVM_INFO ../lib/units/cache_sb.svh(433) @ 2330: uvm_test_top.env.sb [REC_SUREQ] Request: OP=SUREQ_RD ADDR=0x10
4 LOOKUP=SNP_MISS Access to IDX=0x4 WAY=0xx ST= TAG=0xx DAT=0xx
382 UVM_INFO ../lib/units/cache_sb_base.svh(195) @ 2410: uvm_test_top.env.sb [SB] no mismatch between RTL and MODEL
at: IDX=4 WAY=x
383 UVM_INFO ../lib/units/cache_sb.svh(572) @ 2410: uvm_test_top.env.sb [PASS_SUREQ] Report: memory remain, no chan
ge
    
```

Hình 3-34: Kết quả đối chiếu của ví dụ

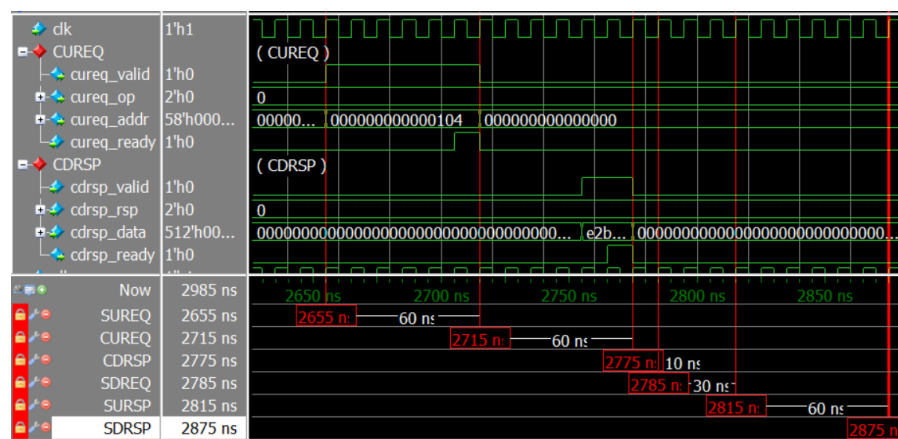
3.2.4.5 Snoop Bus đọc vùng địa chỉ có tồn tại trong bộ nhớ đệm L2

Chi tiết kịch bản như sau:

- Snoop Bus gửi yêu cầu đọc dữ liệu.
- Bản sao của địa chỉ yêu cầu có tồn tại trong bộ nhớ đệm L2.
- Trạng thái của dữ liệu là Migrated.

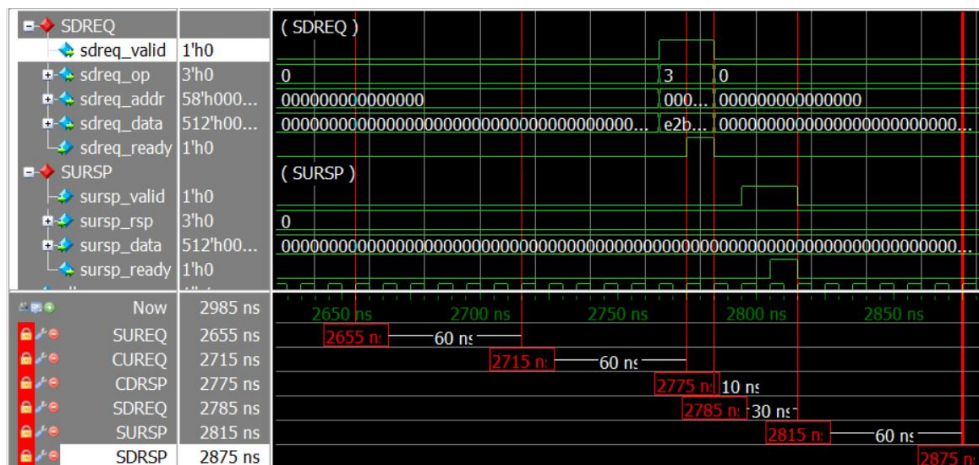
Hành vi của thiết kế được thể hiện như sau:

- Bộ nhớ đệm L2 gửi yêu cầu đọc dữ liệu cho bộ nhớ đệm L1.



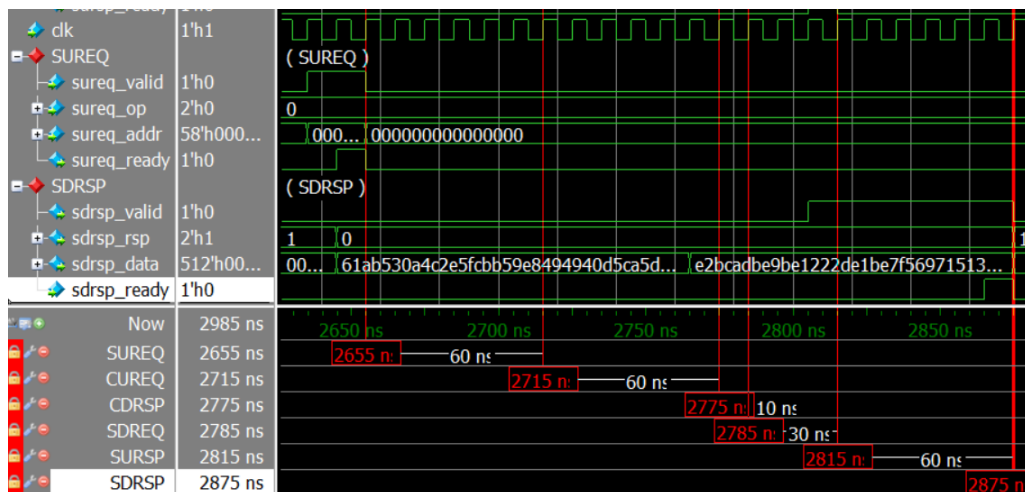
Hình 3-35: Dạng sóng trên kênh CUREQ và CDRSP của ví dụ

- Sau khi nhận phản hồi từ bộ nhớ đệm L1, bộ nhớ đệm L2 gửi Write Back dữ liệu vừa nhận được xuống Snoop Bus.



Hình 3-36: Dạng sóng trên kênh SDREQ và SURSP của ví dụ

- Bộ nhớ đệm L2 gửi dữ liệu cho Snoop Bus.



Hình 3-37: Dạng sóng trên kênh SUREQ và SDRSP của ví dụ

- Bộ nhớ đệm L2 cập nhập dữ liệu mới vào bản sao.

```

402 UVM_INFO ../lib/units/cache_sb.svh(432) @ 2650: uvm_test_top.env.sb [SB] ----- SUREQ -----
403 UVM_INFO ../lib/units/cache_sb.svh(433) @ 2650: uvm_test_top.env.sb [REC_SUREQ] Request: OP=SUREQ_RD ADDR=0x10
4 LOOKUP=HIT Access to IDX=0x4 WAY=0x0 ST=MIGRATED TAG=0x1 DAT=0x61ab530a4c2e5fcb59e84940d5ca5dba7c557b
0283fc1f0c1aba6d06ee5f1f5593ca792c6071a963d81609448b05c90514d0e3fd8d90e6c59eb1e8509e1ef
404 UVM_INFO ../lib/units/cache_sb_base.svh(195) @ 2880: uvm_test_top.env.sb [SB] no mismatch between RTL and MODEL
at: IDX=4 WAY=0
405 UVM_INFO ../lib/units/cache_sb.svh(550) @ 2880: uvm_test_top.env.sb [PASS_SUREQ] Report:
406 UVM_INFO ../lib/units/cache_sb.svh(554) @ 2880: uvm_test_top.env.sb [SB] ST: SHARED <-- MIGRATED
407 UVM_INFO ../lib/units/cache_sb.svh(556) @ 2880: uvm_test_top.env.sb [SB] TAG: 0x1(remain)
408 UVM_INFO ../lib/units/cache_sb.svh(562) @ 2880: uvm_test_top.env.sb [SB] DATA: 0xe2bcadbe9be1222de1be7f56971513
0f016f343320ff1e2273ac6a52c2782a6c223a2f4b6115f13913616d3db627ff49a92b6bfcbe170bf9ad03dea0dad700e <-- 0x61ab530
a4c2e5fcb59e84940d5ca5dba7c557b0283fc1f0c1aba6d06ee5f1f5593ca792c6071a963d81609448b05c90514d0e3fd8d90e6c59eb1
e8509e1ef
409 UVM_INFO ../lib/units/cache_sb.svh(576) @ 2880: uvm_test_top.env.sb [SB] -----
    
```

Hình 3-38: Kết quả đối chiếu của ví dụ

3.2.5 Mô phỏng kịch bản không mong muốn

Dưới đây là một vài trường không mong muốn dựa theo chuẩn MESI protocol và các tính năng mở rộng trong bài luận:

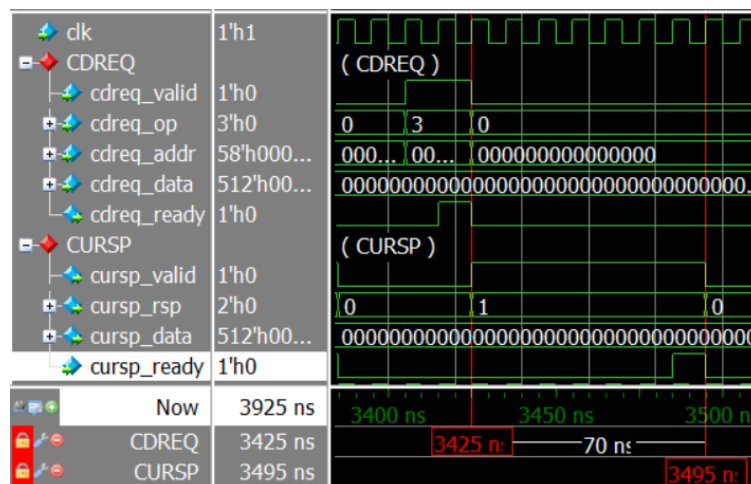
3.2.5.1 Bộ nhớ đệm L1 gửi Mark Dirty cho vùng địa chỉ không tồn tại trong bộ nhớ đệm L2

Chi tiết kịch bản như sau:

- Bộ nhớ đệm L1 gửi thông báo Mark Dirty.
- Bản sao của địa chỉ không tồn tại trong bộ nhớ đệm L2.

Hành vi của thiết kế được thể hiện như sau:

- Bộ nhớ đệm L2 gửi phản hồi lỗi.



Hình 3-39: Dạng sóng trên kênh CDREQ và CURSP của ví dụ

- Dữ liệu trong bộ nhớ không thay đổi.

```

446 UVM_INFO ../lib/units/cache_sb.svh(117) @ 3420: uvm_test_top.env.sb [SB] ----- CDREQ -----
447 UVM_INFO ../lib/units/cache_sb.svh(118) @ 3420: uvm_test_top.env.sb [REC_CDREQ] Request: OP=CDREQ_MD ADDR=0x20
5 DAT=0x0 Lookup=FILL_INV_BLK Access to: IDX=0x5 WAY=0x1 ST=INVALID TAG=0x0 DAT=0x0
448 UVM_WARNING ../lib/units/cache_sb_base.svh(128) @ 3420: uvm_test_top.env.sb [CDREQ_CORNER] OP=CDREQ_MD LOOKUP=
FILL_INV_BLK ST=INVALID
449 UVM_INFO ../lib/units/cache_sb_base.svh(195) @ 3500: uvm_test_top.env.sb [SB] no mismatch between RTL and MODEL
at: IDX=5 WAY=1
450 UVM_INFO ../lib/units/cache_sb.svh(343) @ 3500: uvm_test_top.env.sb [PASS_CDREQ] Report:
451 UVM_INFO ../lib/units/cache_sb.svh(345) @ 3500: uvm_test_top.env.sb [SB] ST=INVALID(remain)
452 UVM_INFO ../lib/units/cache_sb.svh(349) @ 3500: uvm_test_top.env.sb [SB] TAG=0x0(remain)
453 UVM_INFO ../lib/units/cache_sb.svh(353) @ 3500: uvm_test_top.env.sb [SB] DATA=0x0(remain)
454 UVM_INFO ../lib/units/cache_sb.svh(365) @ 3500: uvm_test_top.env.sb [SB] -----

```

Hình 3-40: Kết quả đối chiếu của ví dụ

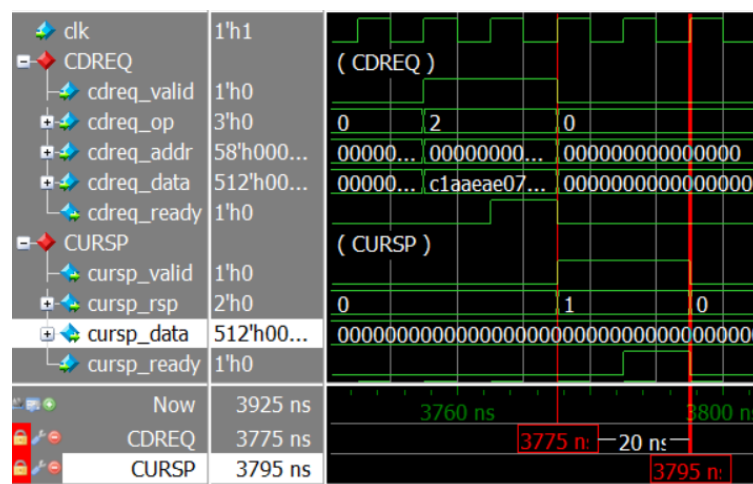
3.2.5.2 Bộ nhớ đệm L1 thực hiện Write Back xuống vùng địa chỉ có trạng thái không phải là Migrated

Chi tiết kịch bản như sau:

- Bộ nhớ đệm L1 gửi yêu cầu Write Back vào vùng địa chỉ.
- Bản sao của địa chỉ có tồn tại trong bộ nhớ đệm L2.
- Trạng thái của dữ liệu là Exclusive.

Hành vi của thiết kế được thể hiện như sau:

- Bộ nhớ đệm L2 gửi phản hồi lỗi.



Hình 3-41: Dạng sóng trên kênh CDREQ và CURSP của ví dụ

- Dữ liệu trong bộ nhớ không thay đổi.

```

474 UVM_INFO ../lib/units/cache_sb.svh(117) @ 3770: uvm_test_top.env.sb [SB] ----- CDREQ -----
475 UVM_INFO ../lib/units/cache_sb.svh(118) @ 3770: uvm_test_top.env.sb [REC_CDREQ] Request: OP=CDREQ_WB ADDR=0x30
5 DAT=0xc1aaeae0736e6da661866dbb4ffce242d76cf3401a3d092b4ec144690052aa708edb350b99635077bb679fdadac32477a88691
d5d6c5843fd7f5ab68acf9506 Lookup=HIT Access to: IDX=0x5 WAY=0x1 ST=EXCLUSIVE TAG=0x3 DAT=0x8b2945ae45853c2
cc72b023794cbe510308fbca3be73b9a65db35f742c24a996af4bd9239ec7552332309696c938a74243fb3a8f587f96838fe11a548408e92
e
476 UVM_WARNING ../lib/units/cache_sb_base.svh(128) @ 3770: uvm_test_top.env.sb [CDREQ_CORNER] OP=CDREQ_WB LOOKUP=
HIT ST=EXCLUSIVE
477 UVM_INFO ../lib/units/cache_sb_base.svh(195) @ 3800: uvm_test_top.env.sb [SB] no mismatch between RTL and MODEL
at: IDX=5 WAY=1
478 UVM_INFO ../lib/units/cache_sb.svh(343) @ 3800: uvm_test_top.env.sb [PASS_CDREQ] Report:
479 UVM_INFO ../lib/units/cache_sb.svh(345) @ 3800: uvm_test_top.env.sb [SB] ST=EXCLUSIVE(remain)
480 UVM_INFO ../lib/units/cache_sb.svh(349) @ 3800: uvm_test_top.env.sb [SB] TAG=0x3(remain)
481 UVM_INFO ../lib/units/cache_sb.svh(353) @ 3800: uvm_test_top.env.sb [SB] DATA=0x8b2945ae45853c2cc72b023794cbe51
0308fbca3be73b9a65db35f742c24a996af4bd9239ec7552332309696c938a74243fb3a8f587f96838fe11a548408e92e(remain)

```

Hình 3-42: Kết quả đối chiếu của ví dụ

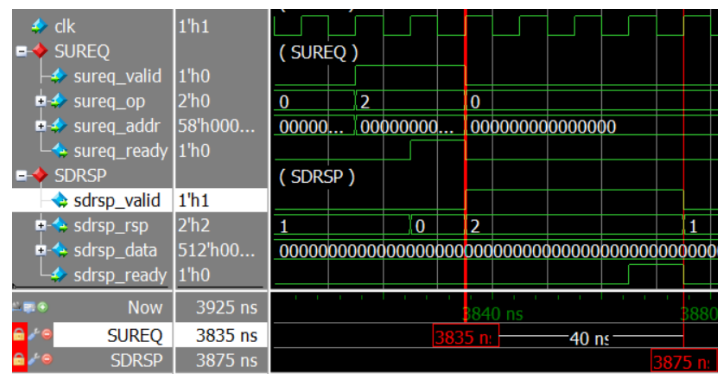
3.2.5.3 Snoop Bus yêu cầu xóa bản sao chỉ tồn tại trong bộ nhớ đệm L2.

Chi tiết kịch bản như sau:

- Snoop Bus gửi yêu cầu xóa bản sao vùng địa chỉ.
- Bản sao của địa chỉ có tồn tại trong bộ nhớ đệm L2.
- Trạng thái của dữ liệu là Exclusive.

Hành vi của thiết kế được thể hiện như sau:

- Bộ nhớ đệm L2 gửi phản hồi lỗi.



Hình 3-43: Dạng sóng trên kênh SUREQ và SDRSP của ví dụ

- Dữ liệu trong bộ nhớ không thay đổi.

```

484 UVM_INFO ../lib/units/cache_sb.svh(432) @ 3830: uvm_test_top.env.sb [SB] ----- SUREQ -----
485 UVM_INFO ../lib/units/cache_sb.svh(433) @ 3830: uvm_test_top.env.sb [REC_SUREQ] Request: OP=SUREQ_INV ADDR=0x3
05 LOOKUP=HIT Access to IDX=0x5 WAY=0x1 ST=EXCLUSIVE TAG=0x3 DAT=0x8b2945ae45853c2cc72b023794cbe510308fbca
3be73b9a65db35f742c24a996af4bd9239ec7552332309696c938a74243fb3a8f587f96838fe11a548408e92e
486 UVM_WARNING ../lib/units/cache_sb_base.svh(138) @ 3830: uvm_test_top.env.sb [SUREQ_CORNER] OP=SUREQ_INV LOOKUP
=HIT ST=EXCLUSIVE
487 UVM_INFO ../lib/units/cache_sb_base.svh(195) @ 3880: uvm_test_top.env.sb [SB] no mismatch between RTL and MODEL
at: IDX=5 WAY=1
488 UVM_INFO ../lib/units/cache_sb.svh(550) @ 3880: uvm_test_top.env.sb [PASS_SUREQ] Report:
489 UVM_INFO ../lib/units/cache_sb.svh(552) @ 3880: uvm_test_top.env.sb [SB] ST: EXCLUSIVE(remain)
490 UVM_INFO ../lib/units/cache_sb.svh(556) @ 3880: uvm_test_top.env.sb [SB] TAG: 0x3(remain)
491 UVM_INFO ../lib/units/cache_sb.svh(560) @ 3880: uvm_test_top.env.sb [SB] DATA: 0x8b2945ae45853c2cc72b023794cbe5
10308fbca3be73b9a65db35f742c24a996af4bd9239ec7552332309696c938a74243fb3a8f587f96838fe11a548408e92e(remain)
492 UVM_INFO ../lib/units/cache_sb.svh(576) @ 3880: uvm_test_top.env.sb [SB] -----
    
```

Hình 3-44: Kết quả đối chiếu của ví dụ

3.2.6 Kết quả và đánh giá

Sau khi chạy các Test để kiểm tra tính năng của MESI protocol (cùng với phần mở rộng trong khóa luận) và các trường hợp không hợp lệ, các thống kê sẽ được tạo ra để tổng kết quá trình kiểm thử.

Kết quả Coverage trong bài Test về MESI protocol như sau:

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/cache_units_pkg/cache_cov_cdreq_c		50.0%				
TYPE CG_CDREQ_MESI_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_CDREQ_MESI_COV::CP_STATE_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_CDREQ_MESI_COV::CP_CDREQ_OP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_CDREQ_MESI_COV::CP_LOOKUP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_CDREQ_MESI_COV::CP_CUREQ_OP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_CDREQ_MESI_COV::CP_CDRSP_RSP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_CDREQ_MESI_COV::CP_SDRSP_OP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_CDREQ_MESI_COV::CP_SURSP_RSP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_CDREQ_MESI_COV::CP_CURSP_RSP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CROSS CG_CDREQ_MESI_COV::CROSS_SDRSP_OP_X_SURSP_RSP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CROSS CG_CDREQ_MESI_COV::CROSS_CDREQ_OP_X_LOOKUP_X_STATE	cache_cov_...	100.0%	100	100.0%	✓	✓
INST \cache_units_pkg::cache_cov_cdreq_c::CG_CDREQ_MESI_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
TYPE CG_CDREQ_CORNER_CASE_COV	cache_cov_...	0.0%	100	0.0%	✗	✓
/cache_units_pkg/cache_cov_sureq_c		50.0%				
TYPE CG_SUREQ_MESI_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_SUREQ_MESI_COV::CP_STATE_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_SUREQ_MESI_COV::CP_SUREQ_OP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_SUREQ_MESI_COV::CP_LOOKUP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_SUREQ_MESI_COV::CP_CUREQ_OP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_SUREQ_MESI_COV::CP_CDRSP_RSP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_SUREQ_MESI_COV::CP_SDRSP_OP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CVP CG_SUREQ_MESI_COV::CP_SURSP_RSP_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
CROSS CG_SUREQ_MESI_COV::CROSS_SUREQ_OP_X_LOOKUP_X_STATE_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
INST \cache_units_pkg::cache_cov_sureq_c::CG_SUREQ_MESI_COV	cache_cov_...	100.0%	100	100.0%	✓	✓
TYPE CG_SUREQ_CONER_CASE_COV	cache_cov_...	0.0%	100	0.0%	✗	✓

Hình 3-45: Kết quả Coverage trong bài Test về MESI protocol

Kết quả thống kê của Scoreboard trong bài Test về MESI protocol như sau:

```
[c] Complete test
[SB] no mismatch between RTL and MODEL at all blocks
[SB] ---- SUMMARY CDREQ ----
[SB] number CDREQ transaction: 52
[SB] number CDREQ functional case: 52
[SB] number CDREQ corner case: 0
[SB] number CDREQ hit: 18
[SB] number CDREQ fill: 22
[SB] number CDREQ evict: 12
[SB] hit rate CDREQ: 34
[SB] -----
[SB] ---- SUMMARY SUREQ ----
[SB] number SUREQ transaction: 13
[SB] number SUREQ functional case: 13
[SB] number SUREQ corner case: 0
[SB] number SUREQ hit: 10
[SB] number SUREQ miss: 3
[SB] hit rate SUREQ: 76
[SB] -----
```

Hình 3-46: Kết quả thống kê của Scoreboard trong bài Test về MESI protocol

Dựa theo Coverage và thống kê của Scoreboard trong bài Test về MESI, chúng ta có thể thấy:

- Các trường hợp của MESI đều đã được bao quát (kết quả là 100%).
- Không có lỗi về hành vi xảy ra.
- Các trường hợp không mong muốn sẽ không được kiểm tra trong bài Test này nên kết quả là 0%..

Kết quả Coverage trong bài Test về các trường hợp không mong muốn như sau:

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/cache_units_pkg/cache_cov_cdreq_c		79.8%				
TYPE CG_CDREQ_MESI_COV	cache_cov_...	59.7%	100	59.7%		✓
TYPE CG_CDREQ_CORNER_CASE_COV	cache_cov_...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_CURSP_RSP_COV	cache_cov_...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_L1_FETCH_HIT_MIGRATED_COV	cache_cov_...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_WB_X_HIT_NOT_MIGRATED_COV	cache_cov_...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_MD_X_MISS_COV	cache_cov_...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_WB_X_MISS_COV	cache_cov_...	100.0%	100	100.0%		✓
INST \cache_units_pkg::cache_cov_cdreq_c::CG_CDREQ_CORNER_CASE_COV	cache_cov_...	100.0%	100	100.0%		✓
/cache_units_pkg/cache_cov_sureq_c		61.5%				
TYPE CG_SUREQ_MESI_COV	cache_cov_...	23.1%	100	23.1%		✓
TYPE CG_SUREQ_CONER_CASE_COV	cache_cov_...	100.0%	100	100.0%		✓
CVP CG_SUREQ_CONER_CASE_COV::CP_INV_NOT_HIT_SHARED_COV	cache_cov_...	100.0%	100	100.0%		✓
INST \cache_units_pkg::cache_cov_sureq_c::CG_SUREQ_CONER_CASE_COV	cache_cov_...	100.0%	100	100.0%		✓

Hình 3-47: Kết quả Coverage trong bài Test về các trường hợp không mong muốn

Kết quả thống kê của Scoreboard trong bài Test về các trường hợp không mong muốn như sau:

```
est_c] Complete test
[SB] no mismatch between RTL and MODEL at all blocks
[SB] ---- SUMMARY CDREQ ----
[SB] number CDREQ transaction: 16
[SB] number CDREQ functional case: 7
[SB] number CDREQ corner case: 9
[SB] number CDREQ hit: 2
[SB] number CDREQ fill: 5
[SB] number CDREQ evict: 0
[SB] hit rate CDREQ: 28
[SB] -----
[SB] ---- SUMMARY SUREQ ----
[SB] number SUREQ transaction: 4
[SB] number SUREQ functional case: 1
[SB] number SUREQ corner case: 3
[SB] number SUREQ hit: 1
[SB] number SUREQ miss: 0
[SB] hit rate SUREQ: 100
[SB] -----
```

Hình 3-48: Kết quả thống kê của Scoreboard trong bài Test về các trường hợp không mong muốn

Sau khi có kết quả Coverage của từng Test, chúng ta có thể gộp tất cả thành một Coverage tổng quát như sau:

Name	Class Type	Coverage	Goal	% of Goal	Status	Includ
/cache_units_pkg/cache_cov_cdreq_c		100.0%				
TYPE CG_CDREQ_MESI_COV	cache_cov...	100.0%	100	100.0%		✓
TYPE CG_CDREQ_CORNER_CASE_COV	cache_cov...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_CURSP_RSP_COV	cache_cov...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_L1_FETCH_HIT_MIGRATED_COV	cache_cov...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_WB_X_HIT_NOT_MIGRATED_COV	cache_cov...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_MD_X_MISS_COV	cache_cov...	100.0%	100	100.0%		✓
CVP CG_CDREQ_CORNER_CASE_COV::CP_WB_X_MISS_COV	cache_cov...	100.0%	100	100.0%		✓
INST \cache_units_pkg::cache_cov_cdreq_c::CG_CDREQ_CORNER_CASE_COV		100.0%	100	100.0%		✓
/cache_units_pkg/cache_cov_sureq_c		100.0%				
TYPE CG_SUREQ_MESI_COV	cache_cov...	100.0%	100	100.0%		✓
TYPE CG_SUREQ_CONER_CASE_COV	cache_cov...	100.0%	100	100.0%		✓
CVP CG_SUREQ_CONER_CASE_COV::CP_INV_NOT_HIT_SHARED_COV	cache_cov...	100.0%	100	100.0%		✓
INST \cache_units_pkg::cache_cov_sureq_c::CG_SUREQ_CONER_CASE_COV		100.0%	100	100.0%		✓

Hình 3-49: Tổng kết Coverage trong tất cả Test

Từ các thông số Coverage trên, chúng ta có thể thấy các bài Test đã kiểm tra đầy đủ các trường hợp của MESI protocol cũng như các trường hợp không mong muốn.

Với kết quả Coverage là 100% với các trường hợp của MESI protocol và Scoreboard không phát hiện lỗi hành vi trong quá trình mô phỏng, chúng ta có thể đánh giá rằng thiết kế đã hoạt động đúng như yêu cầu của bài luận đề ra.

CHƯƠNG 4: KẾT LUẬN

4.1 Đánh giá kết quả

4.1.1 Ưu điểm

Bài luận đã hoàn thành trong việc thiết kế bộ nhớ đệm riêng L2 (Private Cache) nằm bên ngoài Core, giúp bộ nhớ đệm L1 giao tiếp với hệ thống thông qua bộ nhớ đệm L2. Thiết kế đã đáp ứng các yêu cầu ban đầu như:

- Thực hiện giao thức MESI protocol trong mô hình Coherence.
- Mở rộng giao thức MESI như hỗ trợ thêm trạng thái Migrated và loại yêu cầu Mark Dirty để đảm bảo tính nhất quán của bộ nhớ đệm L1.
- Đảm bảo tính bao hàm (Inclusivity Property) trên hai bộ nhớ đệm L1 và L2.
- Phương pháp áp xạ bộ nhớ 4-Ways Set Associative và thuật toán thay thế PLRU hoạt động đúng với yêu cầu
- Đã xây dựng được môi trường UVM có thể gửi và nhận các tín hiệu trên Interface, Scoreboard để kiểm tra hành vi của thiết kế và Coverage để đảm bảo đã bao phủ toàn bộ kịch bản có thể xảy ra.
- Hỗ trợ xuất ra dạng sóng và Log file để theo dõi hoạt động của thiết kế.

4.1.2 Nhược điểm

Ngoài các mục tiêu đã đạt được, thiết kế còn vài hạn chế sau:

- Chưa có thuật toán theo dõi sự tồn tại của dữ liệu trong bộ nhớ đệm L1, dẫn đến việc khởi tạo các lệnh Invalidate không cần thiết đến bộ nhớ đệm L1.
- Thiết kế chưa có bộ phân xử (Arbitor) để đảm bảo việc khởi tạo các lệnh phát sinh một cách trình tự, tránh trường hợp xung đột giữa bộ xử lý yêu cầu từ bộ nhớ đệm L1 và Snoop Bus khi cùng muốn gửi yêu cầu trên kênh SDREQ hay CUREQ.

4.2 Định hướng phát triển

Ngoài các nhược điểm trên có thể phát triển thêm như sau:

- Phát triển thuật toán thay thế, để tăng tỉ lệ Hit và giảm tài nguyên cần thiết
- Phát triển random Test dựa trên Constrain-random để kiểm tra được bao quát hơn
- Hỗ trợ Coherence protocol phức tạp hơn như MOESI hay MESIF.

TÀI LIỆU THAM KHẢO

- [1] "Cache coherence," *Wikipedia*. May 27, 2025. Accessed: July 25, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Cache_coherence&oldid=1292490736
- [2] "Cache Coherence," GeeksforGeeks. Accessed: July 25, 2025. [Online]. Available: <https://www.geeksforgeeks.org/operating-systems/cache-coherence/>
- [3] R. Ware, "L1, L2, and L3 Cache: What's the Difference?," How-To Geek. Accessed: July 25, 2025. [Online]. Available: <https://www.howtogeek.com/891526/l1-vs-l2-vs-l3-cache/>
- [4] "ECE 485/585 Cache Coherence", Portland State University Mark G. Faust, Microprocessor System Design.
- [5] "gem5: CHI." Accessed: July 25, 2025. [Online]. Available: https://www.gem5.org/documentation/general_docs/ruby/CHI/
- [6] "chapter5 - set associative caches", Morgan Kaufmann, COMPUTER ORGANIZATION AND DESIGN.
- [7] team, "Cache Algorithms: FIFO vs. LRU vs. LFU - A Comprehensive Guide," AlgoCademy Blog. Accessed: July 25, 2025. [Online]. Available: <https://algotcademy.com/blog/cache-algorithms-fifo-vs-lru-vs-lfu-a-comprehensive-guide/>
- [8] "Pseudo-LRU," *Wikipedia*. Apr. 25, 2024. Accessed: July 25, 2025. [Online]. Available: <https://en.wikipedia.org/w/index.php?title=Pseudo-LRU&oldid=1220728132>
- [9] "Differentiate between Write Through and Write Back Methods," GeeksforGeeks. Accessed: July 25, 2025. [Online]. Available: <https://www.geeksforgeeks.org/computer-organization-architecture/differentiate-between-write-through-and-write-back-methods/>
- [10] "Difference between Write invalidate protocol & Write Update Protocol (Bus-Snooping)," GeeksforGeeks. Accessed: July 25, 2025. [Online]. Available: <https://www.geeksforgeeks.org/computer-organization-architecture/difference-between-write-invalidate-protocol-write-update-protocol-bus-snooping/>
- [11] "Cache inclusion policy," *Wikipedia*. Jan. 25, 2025. Accessed: July 25, 2025. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Cache_inclusion_policy&oldid=1271773202
- [12] Admin, "UVM Tutorial," ChipVerify. Accessed: July 25, 2025. [Online]. Available: <https://www.chipverify.com/tutorials/uvm?mdrv=www.chipverify.com>
- [13] "Functional Coverage - VLSI Verify." Accessed: July 25, 2025. [Online]. Available: <https://vlsiverify.com/system-verilog/functional-coverage/functional-coverage/>